

SLURM

1ère partie

Lise Jolicoeur, 2024

Quelques mots sur l'ordonnancement

Emergence des clusters

Interconnexion d'unités individuelles

Emergence des clusters

Interconnexion d'unités individuelles



=> 4356 noeuds

Emergence des clusters

Interconnexion d'unités individuelles



=> 4356 noeuds

4320 noeuds <=



Emergence des clusters

Interconnexion d'unités individuelles



=> 4356 noeuds

4320 noeuds <=



=> 864 noeuds

=> 48 noeuds a100

Emergence des clusters

Implique l'orchestration des unités de calculs indépendantes

Emergence des clusters

Implique l'orchestration des unités de calculs indépendantes

« Systèmes distribués »

Distribution des travaux sur les ressources en parallèle

Emergence des clusters

Implique l'orchestration des unités de calculs indépendantes

« Systèmes distribués »

Distribution des travaux sur les ressources en parallèle

Répartition **spatiale** et **temporelle**

Emergence des clusters

Job hétérogènes

Plusieurs sections parallèles

Emergence des clusters

Job hétérogènes

Plusieurs sections parallèles

Emergence du modèle MPI

Echange de données entre les processus



Emergence des clusters

Job hétérogènes

Plusieurs sections parallèles

Emergence du modèle MPI

Echange de données entre les processus

=> Complexification de l'ordonnancement



Architecture des DRMS

Rappels

Rappels

Job manager

Resource manager

Rappels

Job manager

Evolution des batch scheduler

Principe du time slicing

Resource manager

Rappels

Job manager

Evolution des batch scheduler

Principe du time slicing

Resource manager

Ressources grandissantes

Et distribuées

Architecture

Architecture

Leader central

Architecture

Leader central



Architecture

Leader central

Enregistre les « jobs » pour plus tard



Architecture

Leader central

Enregistre les « jobs » pour plus tard

Statut des ressources & jobs



Architecture

Leader central

Enregistre les « jobs » pour plus tard

Statut des ressources & jobs

Historique et statistiques



Architecture

Leader central

Enregistre les « jobs » pour plus tard

Statut des ressources & jobs

Historique et statistiques

Orchestre les jobs sur les ressources



Architecture

Leader central

Enregistre les « jobs » pour plus tard

Statut des ressources & jobs

Historique et statistiques

Orchestre les jobs sur les ressources

Suit le cycle de vie du job



Architecture

Workers distribués

Architecture

Workers distribués



Architecture

Workers distribués

Un par noeud de calcul



Architecture

Workers distribués

Un par noeud de calcul

Fournit état du noeud au leader



Architecture

Workers distribués

Un par noeud de calcul

Fournit état du noeud au leader

Lance les scripts/applications



Architecture

Workers distribués

Un par noeud de calcul

Fournit état du noeud au leader

Lance les scripts/applications

Suivi de l'utilisation et libération des ressources



Architecture

Workers distribués

Un par noeud de calcul

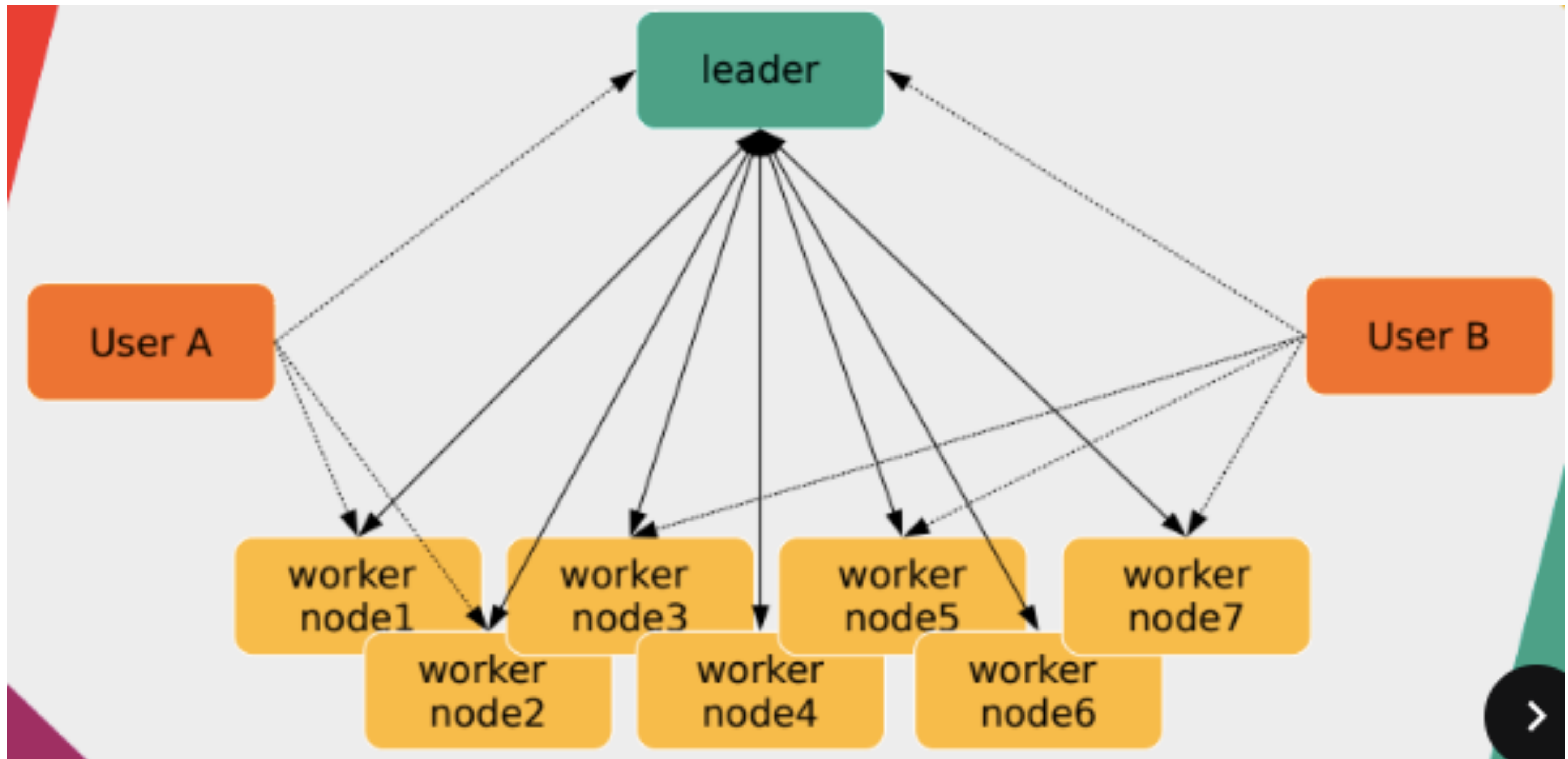
Fournit état du noeud au leader

Lance les scripts/applications

Suivi de l'utilisation et libération des ressources



Architecture



Quelques exemples de DRMS

OAR

OAR



OAR

Batch scheduler développé à Grenoble (CNRS, INRIA, etc)



OAR

Batch scheduler développé à Grenoble (CNRS, INRIA, etc)

Basé sur une base de données (PostgreSQL)



OAR

Batch scheduler développé à Grenoble (CNRS, INRIA, etc)

Basé sur une base de données (PostgreSQL)

Pas de démons sur les noeuds de calculs



OAR

Batch scheduler développé à Grenoble (CNRS, INRIA, etc)

Basé sur une base de données (PostgreSQL)

Pas de démons sur les noeuds de calculs

Utilisé pour le Grid5000

<https://oar.imag.fr/start>



OpenPBS

OpenPBS/Torque



OpenPBS/Torque

Batch scheduler basé sur des plugins ('hooks')



OpenPBS/Torque

Batch scheduler basé sur des plugins ('hooks')

Testé sur plus de 50,000 noeuds



OpenPBS/Torque

Batch scheduler basé sur des plugins ('hooks')

Testé sur plus de 50,000 noeuds

Pas de SPOFs, grande résilience



OpenPBS/Torque

Batch scheduler basé sur des plugins ('hooks')

Testé sur plus de 50,000 noeuds

Pas de SPOFs, grande résilience

<https://www.openpbs.org>



Et pleins d'autres...

Et pleins d'autres...

Spectrum LSF



Et pleins d'autres...

Spectrum LSF



Moab

Et pleins d'autres...

Spectrum LSF



GridEngine



Moab

Et finalement : SLURM

Et finalement : SLURM

Simple Linux Utility for Resource Management



Et finalement : SLURM

Simple Linux Utility for Resource Management

Démarré au LLNL en 2002



Et finalement : SLURM

Simple Linux Utility for Resource Management

Démarré au LLNL en 2002

Continué par SchedMD en 2010

2 anciens développeurs!



Slurm : introduction des notions élémentaires

SLURM : Introduction

SLURM : Introduction

Open Source, écrit en C

Licence GPLv2



SLURM : Introduction

Open Source, écrit en C

Licence GPLv2

Exclusivement sur UNIX

AIX, Linux, BSD, etc



UNIX®
A Standard of The Open Group®

SLURM : Introduction

Open Source, écrit en C

Licence GPLv2

Exclusivement sur UNIX

AIX, Linux, BSD, etc

Utilisé partout, 60% du top500



UNIX[®]
A Standard of The Open Group[®]



SLURM : Introduction

Scalable

10 000+ noeuds, 100M+ coeurs



SLURM : Introduction

Scalable

10 000+ noeuds, 100M+ coeurs

Modulaire

Définition de « plugins »

Spécialisation en fonction du besoin, ajout de fonctionnalités



SLURM : Entités élémentaires

slurmctld

« contrôleur », composant central

SLURM : Entités élémentaires

slurmctld

« contrôleur », composant central

slurmdbd

Comptabilité : jobs, utilisateurs, droits, etc

SLURM : Entités élémentaires

slurmctld

« contrôleur », composant central

slurmdbd

Comptabilité : jobs, utilisateurs, droits, etc

slurmd

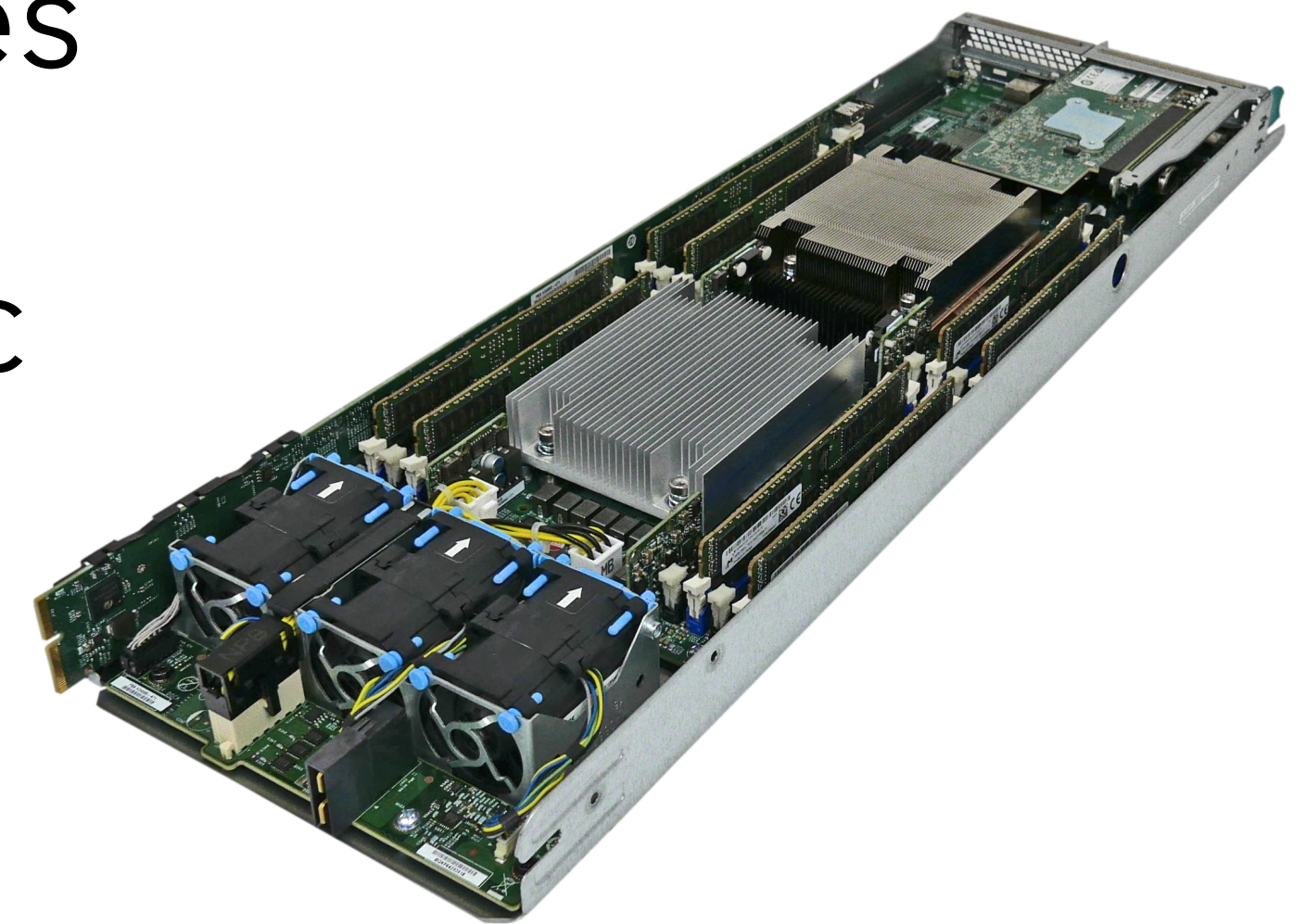
« worker », composant sur chaque noeud

SLURM : Entités élémentaires

Noeud (« Node »)

Unité indépendante fournissant des ressources

Sockets / Cores / Threads, mémoire, GPUs, etc



SLURM : Entités élémentaires

Noeud (« Node »)

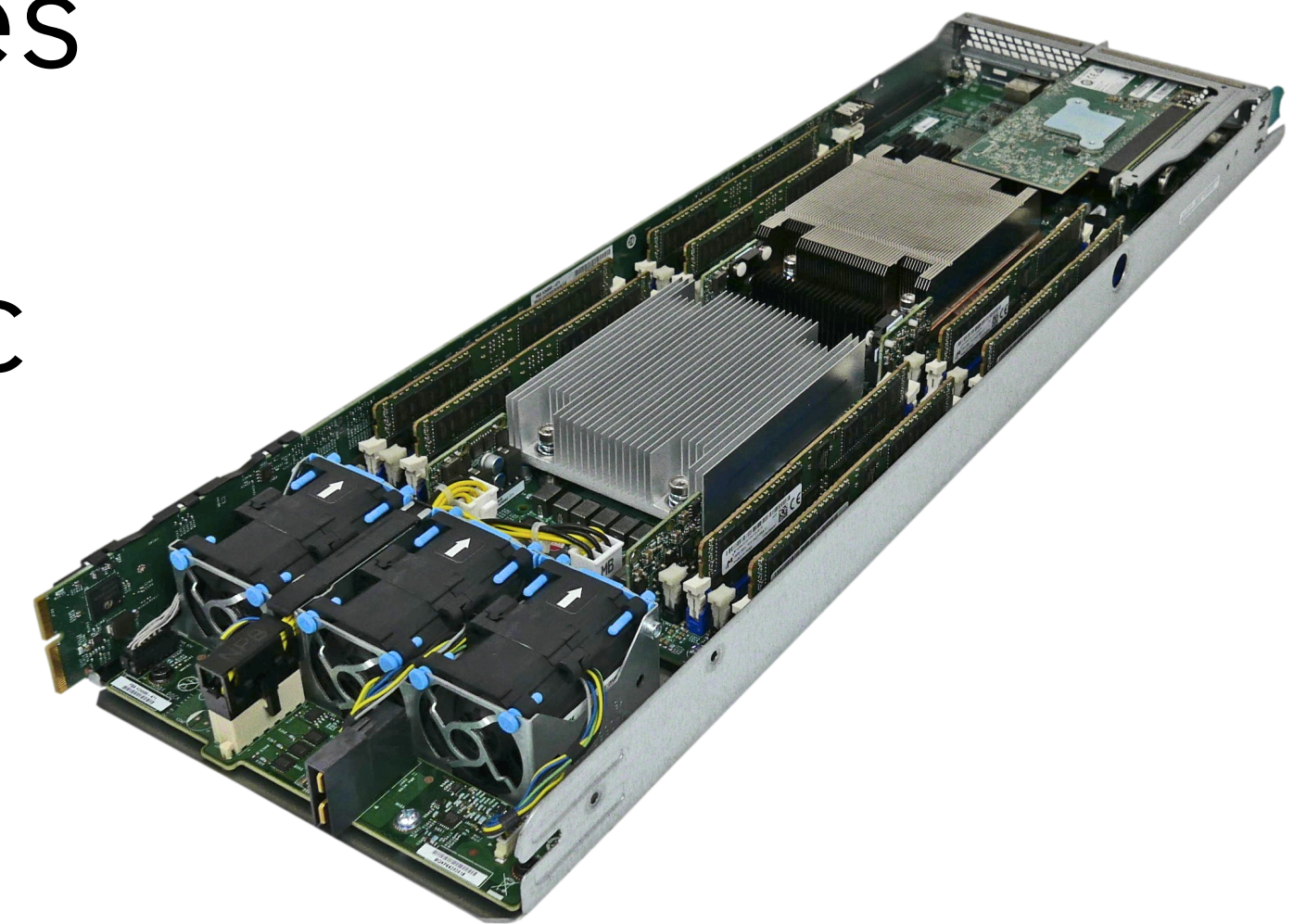
Unité indépendante fournissant des ressources

Sockets / Cores / Threads, mémoire, GPUs, etc

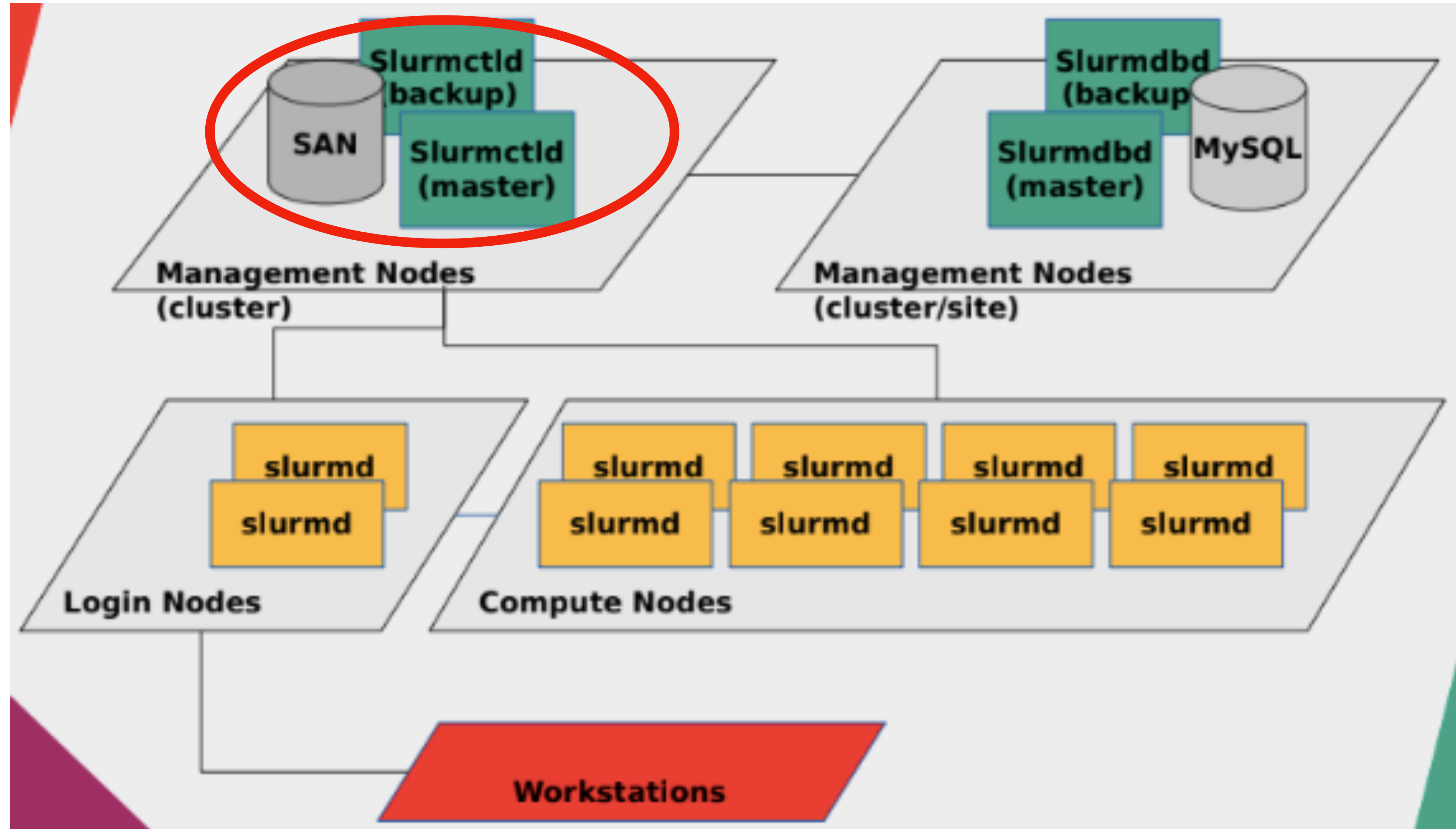
Partition

Ensemble de noeuds homogènes

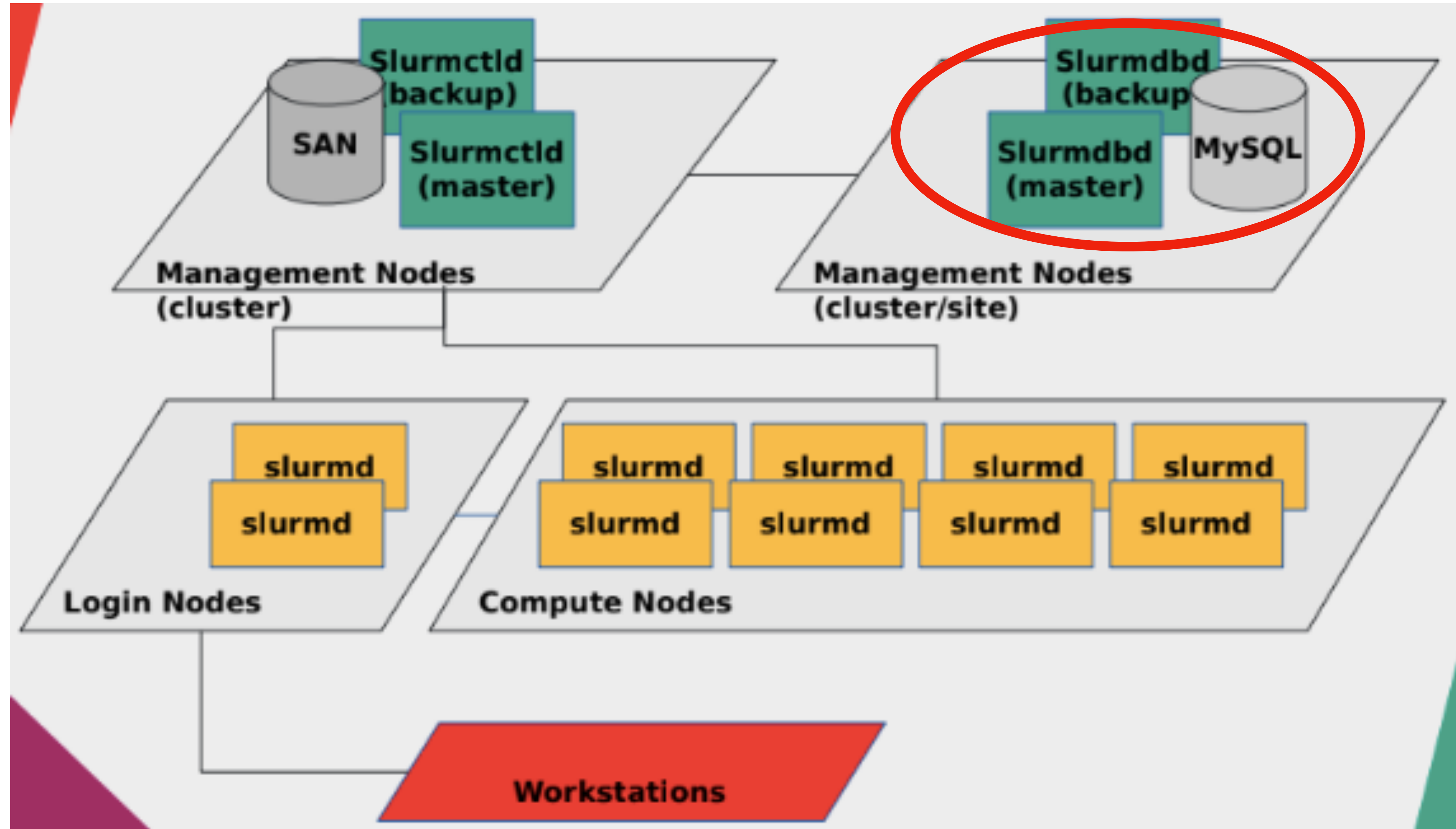
Un job ne tourne que sur 1 partition à la fois



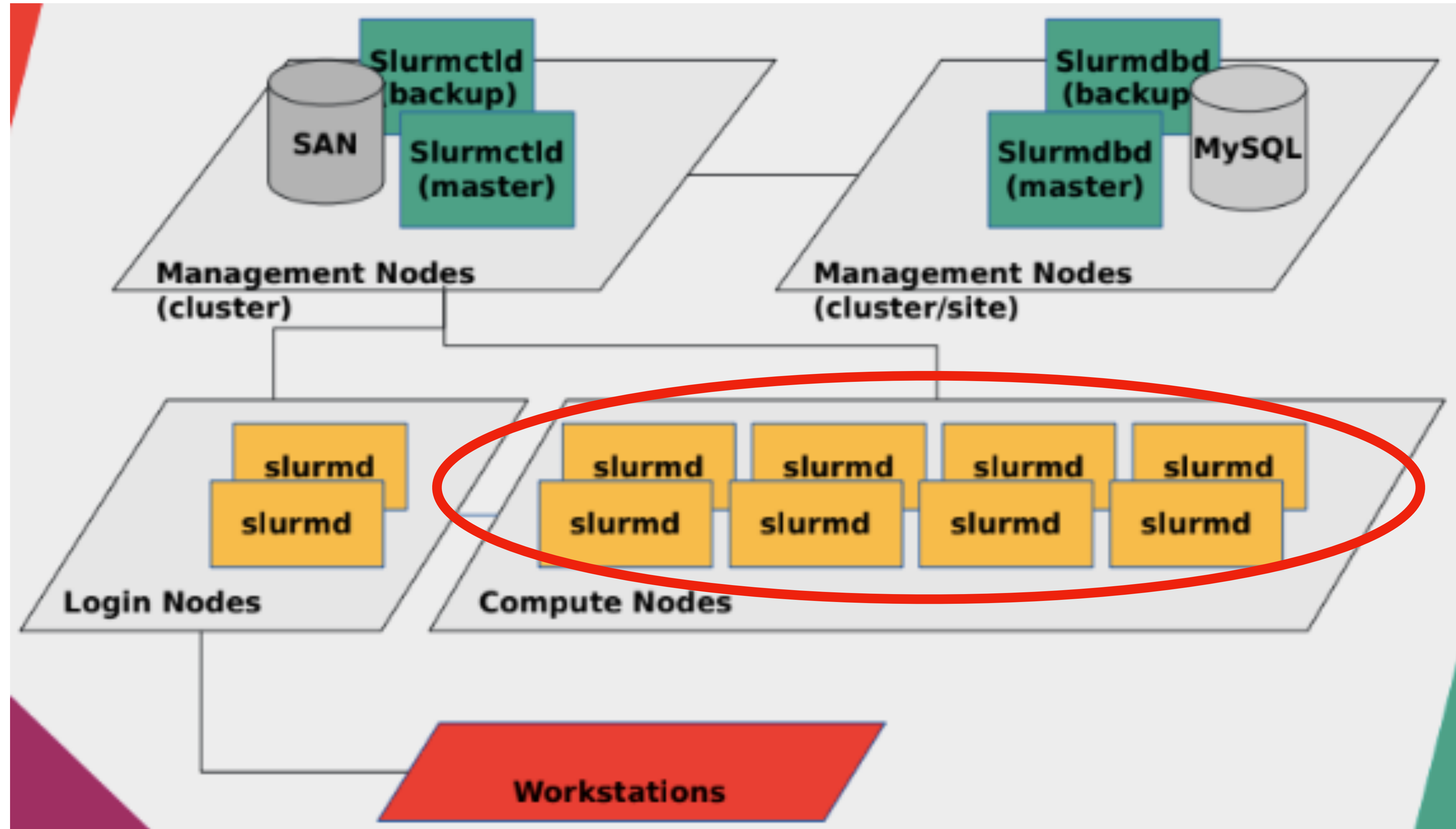
SLURM : Entités élémentaires



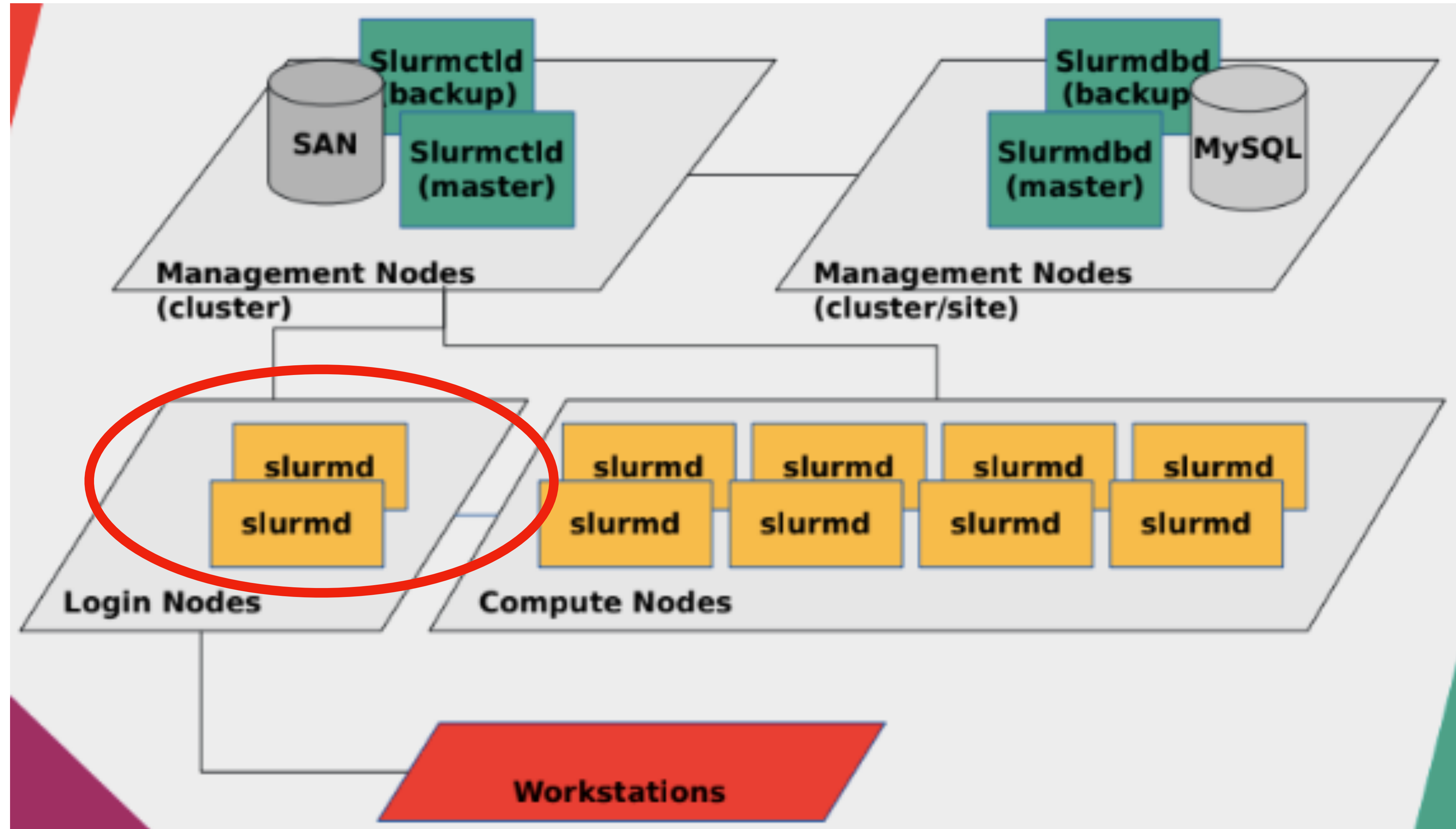
SLURM : Entités élémentaires



SLURM : Entités élémentaires



SLURM : Entités élémentaires



SLURM : Entités élémentaires

Job

SLURM : Entités élémentaires

Job

Demande d'allocation dans une partition

SLURM : Entités élémentaires

Job

Demande d'allocation dans une partition

Ensemble fini de ressources pendant temps défini

Batch (script) ou interactif

SLURM : Entités élémentaires

Job

Demande d'allocation dans une partition

Ensemble fini de ressources pendant temps défini

SLURM : Entités élémentaires

JobStep

SLURM : Entités élémentaires

JobStep

Demande de **sous allocation** de ressources dans un job

SLURM : Entités élémentaires

JobStep

Demande de **sous allocation** de ressources dans un job

Pour une tâche particulière

SLURM : Entités élémentaires

JobStep

Demande de **sous allocation** de ressources dans un job

Pour une tâche particulière

Sous-ensemble ou ensemble des ressources

SLURM : Entités élémentaires

« Step » = étape

Plusieurs steps dans un job

En parallèle sur des ressources différentes, ou séquentiel

SLURM : Entités élémentaires

« Step » = étape

Plusieurs steps dans un job

En parallèle sur des ressources différentes, ou séquentiel

Une analogie ?

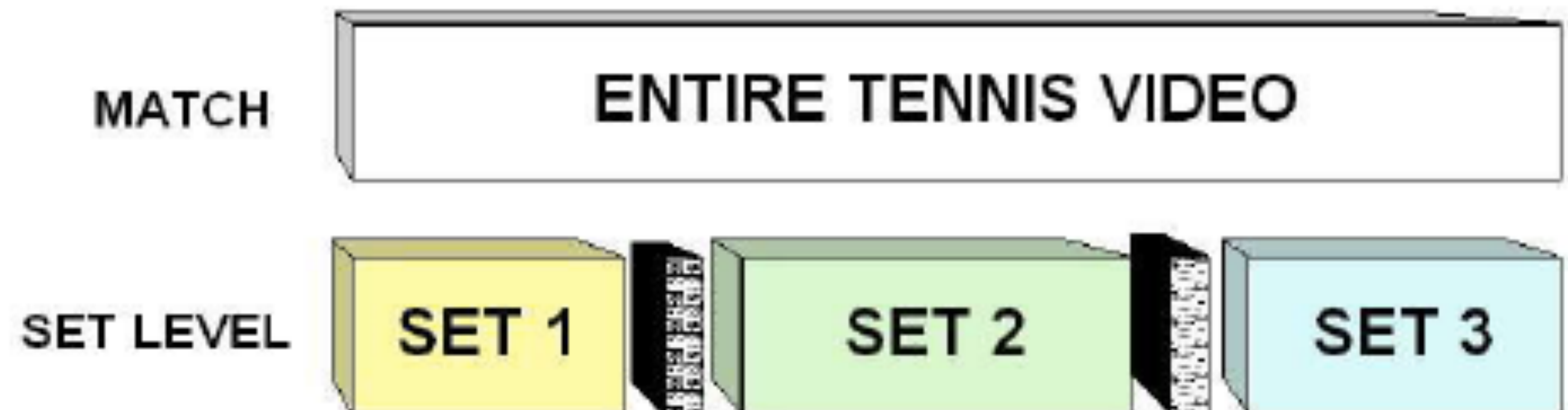
SLURM : Entités élémentaires

« **Step** » = étape

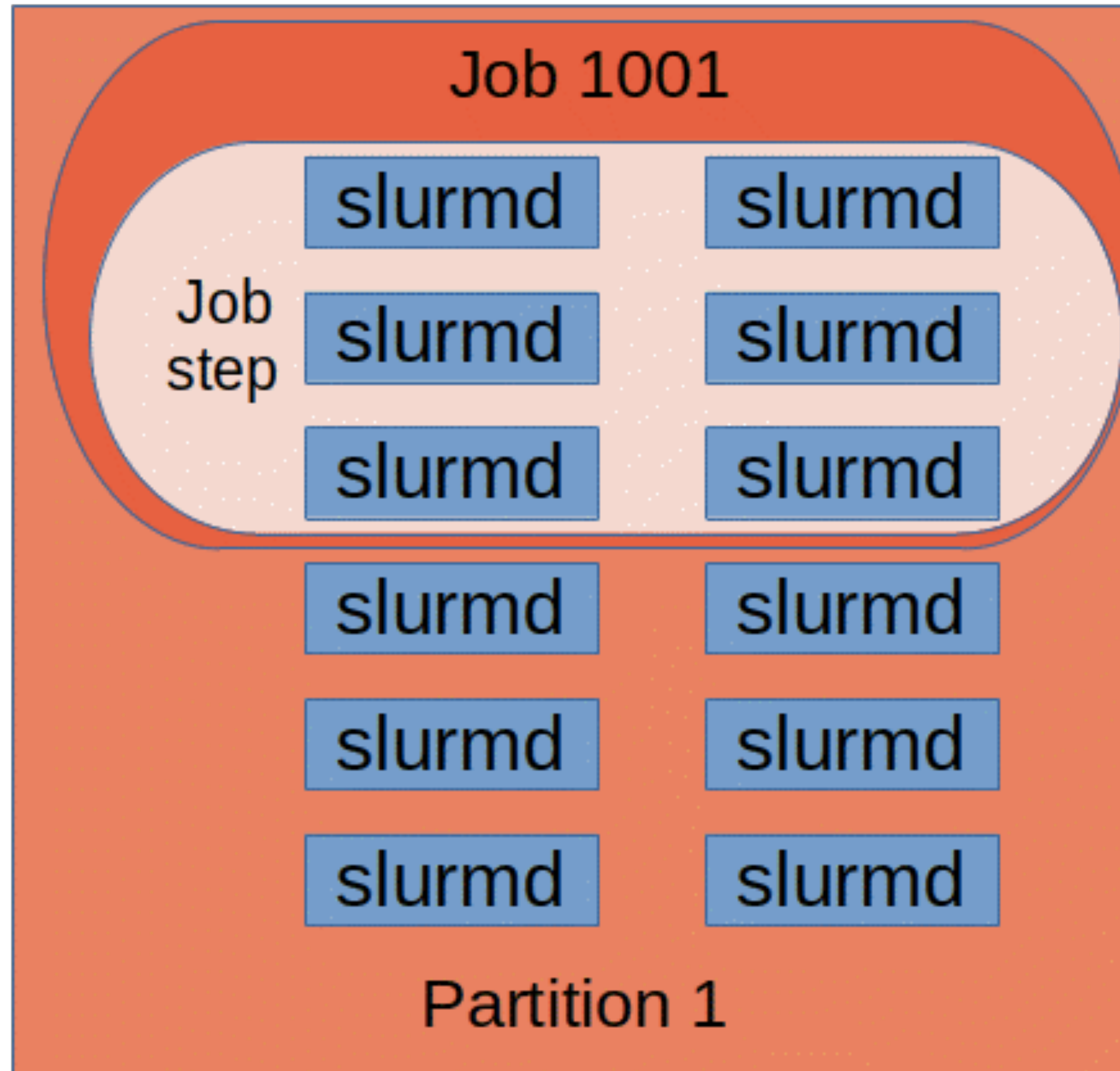
Plusieurs steps dans un job

En parallèle sur des ressources différentes, ou séquentiel

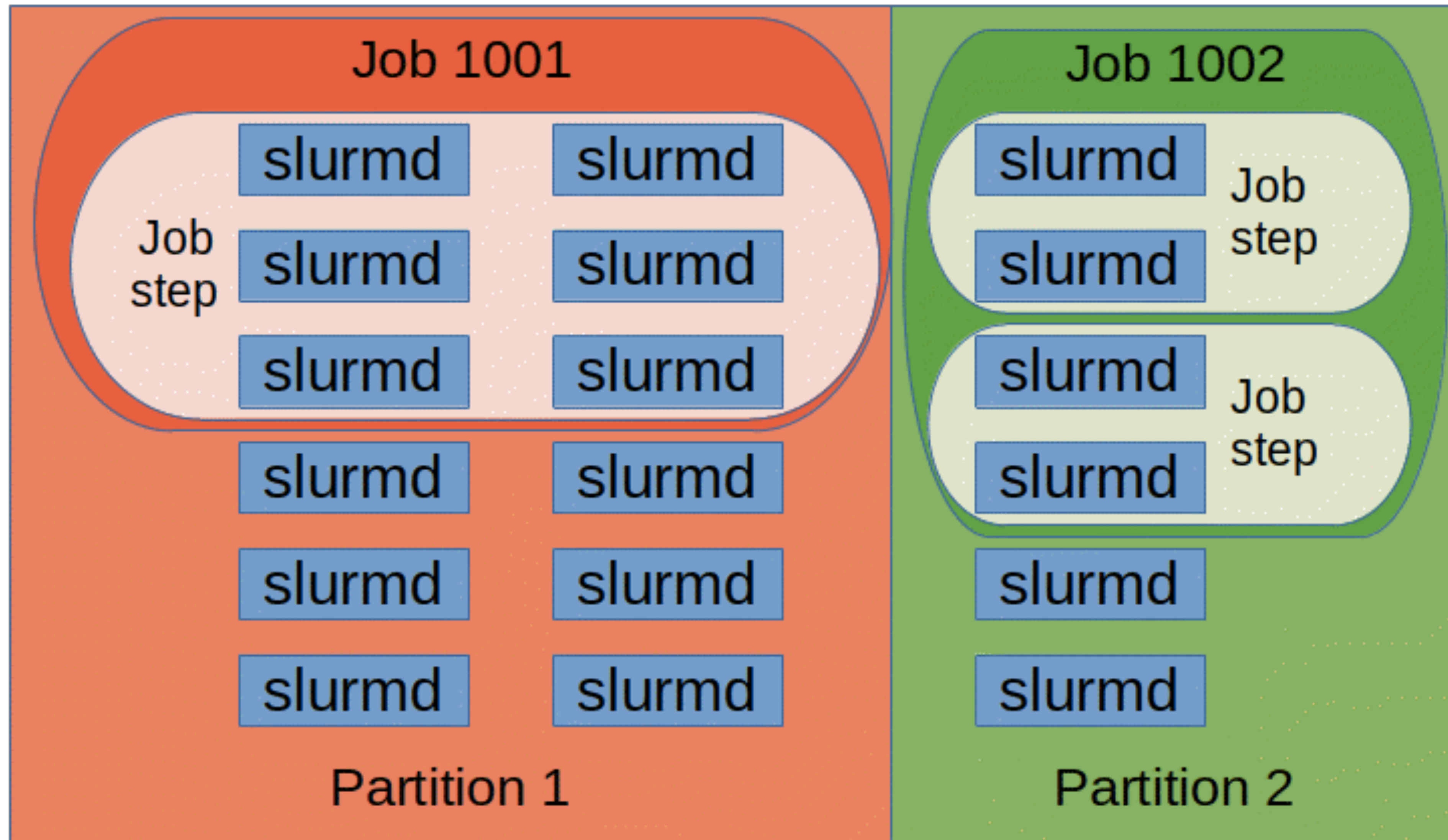
Une analogie ?



SLURM : Entités élémentaires



SLURM : Entités élémentaires



SLURM : Entités élémentaires

Noeud (« Node »)

Unité indépendante fournissant des ressources

SLURM : Entités élémentaires

Noeud (« Node »)

Unité indépendante fournissant des ressources

Peut être dans plusieurs **états (states)**

SLURM : Entités élémentaires

Noeud (« Node »)

Unité indépendante fournissant des ressources

Peut être dans plusieurs **états (states)**

Récupération de l'état : **sinfo**

SLURM : Entités élémentaires

Noeud (« Node »)

Unité indépendante fournissant des ressources

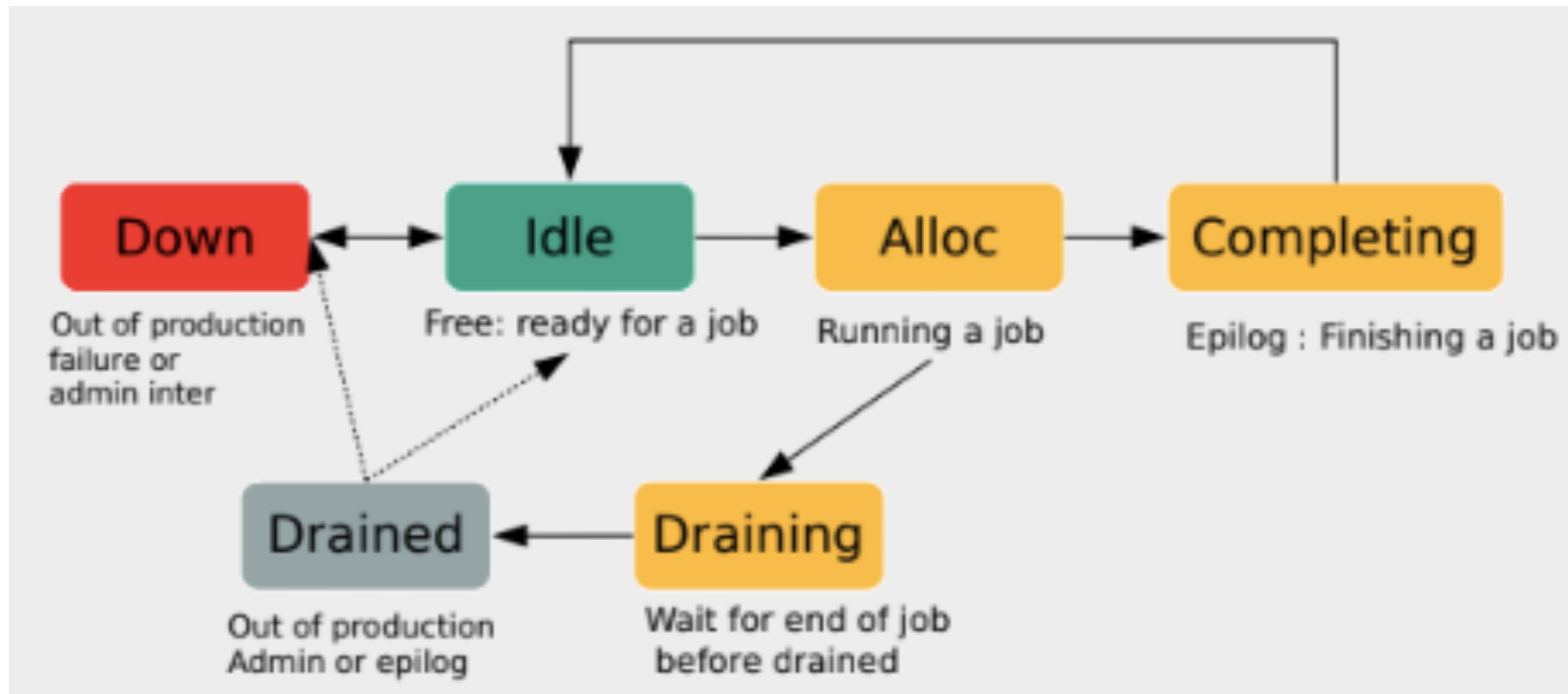
Peut être dans plusieurs **états (states)**

Récupération de l'état : **sinfo**

Modification de l'état : **scontrol update**

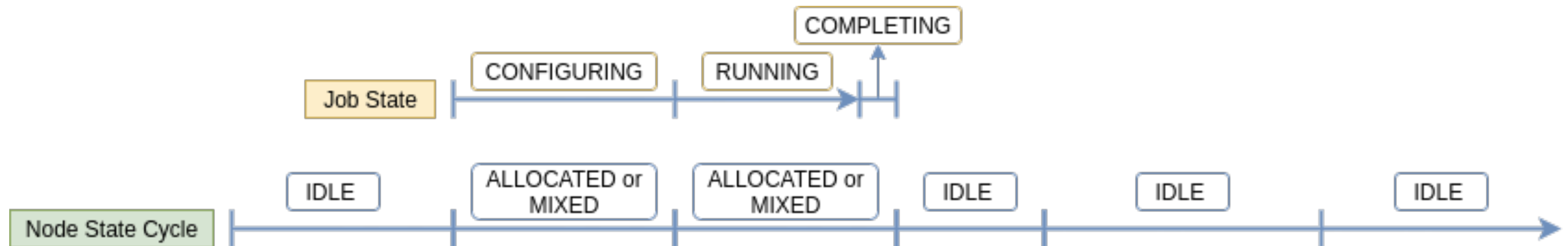
SLURM : Entités élémentaires

Noeud (« Node ») : états



SLURM : Entités élémentaires

Noeud (« Node ») : états



SLURM : Entités élémentaires

Job

Demande d'allocation dans une partition

SLURM : Entités élémentaires

Job

Demande d'allocation dans une partition

Traverse plusieurs **états** dans son **cycle de vie**

SLURM : Entités élémentaires

Job

Demande d'allocation dans une partition

Traverse plusieurs **états** dans son **cycle de vie**

Récupération de l'état : **squeue**

SLURM : Entités élémentaires

Job

Demande d'allocation dans une partition

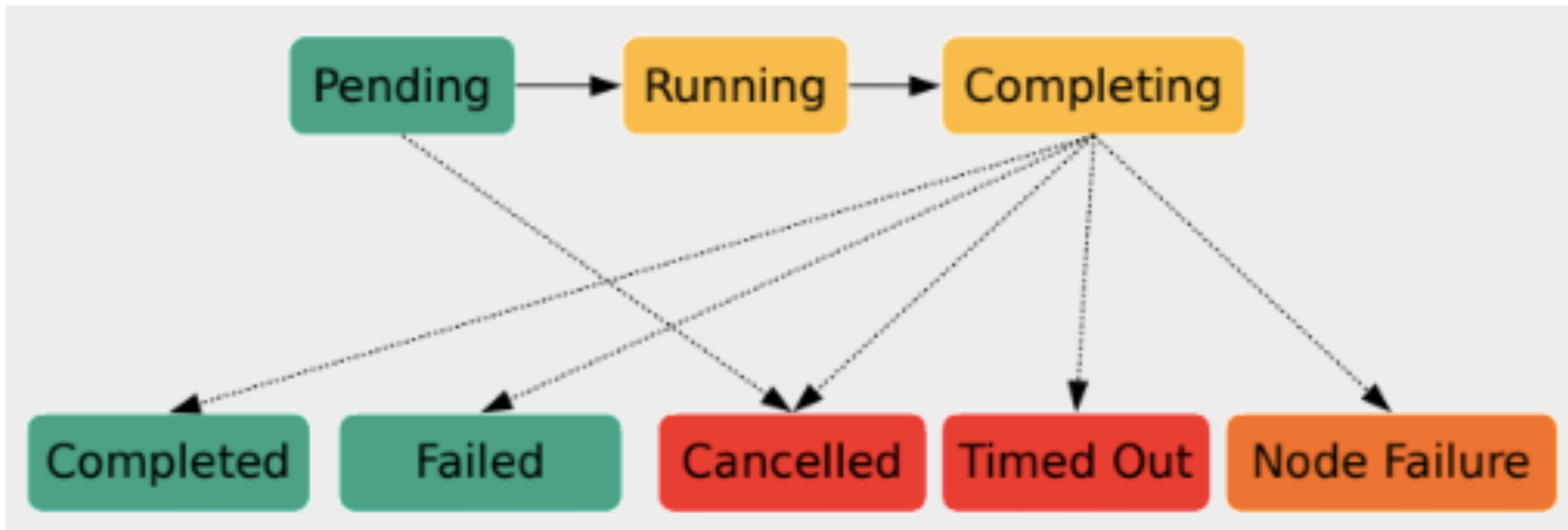
Traverse plusieurs **états** dans son **cycle de vie**

Récupération de l'état : **squeue**

Modification de l'état : A voir en TP!

SLURM : Entités élémentaires

Job : états



SLURM : Configuration

SLURM : Configuration

scontrol

SLURM : Configuration

scontrol

Obtention et modification de la configuration

SLURM : Configuration

scontrol

Obtention et modification de la configuration

Obtention et modification des **états** des éléments

SLURM : Configuration

scontrol

Obtention et modification de la configuration

Obtention et modification des **états** des éléments

sacctmgr

SLURM : Configuration

scontrol

Obtention et modification de la configuration

Obtention et modification des **états** des éléments

sacctmgr

Obtention et modification de la configuration de la BD

SLURM : Information

sinfo

SLURM : Information

sinfo

Information sur l'état des partitions

SLURM : Information

sinfo

Information sur l'état des partitions

squeue

SLURM : Information

sinfo

Information sur l'état des partitions

squeue

Information sur l'état des jobs

SLURM : Information

sacct

SLURM : Information

sacct

Information sur l'exécution de jobs en cours ou passés

SLURM : Information

sacct

Information sur l'exécution de jobs en cours ou passés

sstat

SLURM : Information

sacct

Information sur l'exécution de jobs en cours ou passés

sstat

Information détaillée sur l'exécution de jobs en cours

SLURM : Lancement de jobs

3 commandes de bases :

SLURM : Lancement de jobs

3 commandes de bases :

srun

SLURM : Lancement de jobs

3 commandes de bases :

srun

sbatch

SLURM : Lancement de jobs

3 commandes de bases :

srun

salloc

sbatch

SLURM : Lancement de jobs

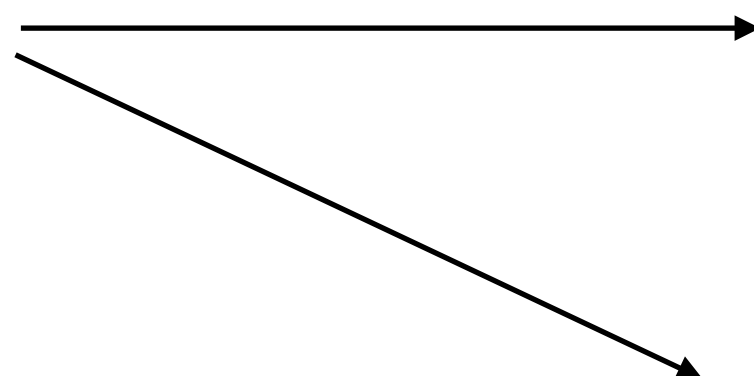
srun

Execution de processus répartis sur des ressources

Typiquement, un programme MPI

Techniquement : création d'un **job step**

2 cas de figure :



- Execution directe
- Execution dans un job existant

SLURM : Lancement de jobs

Execution directe

Création préalable du job

Allocation des ressources

Utilisation, et libération

=> Job avec 1 seul step

Execution dans un job

Ressources déjà allouées

Ne change pas état du job

Plusieurs steps possibles

SLURM : Lancement de jobs

salloc

SLURM : Lancement de jobs

salloc

Demande d'allocation de ressources

SLURM : Lancement de jobs

salloc

Demande d'allocation de ressources

Crée un « job » => assignation d'un ID et de ressources

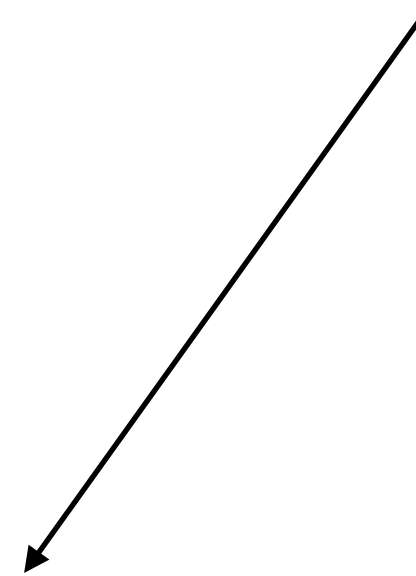
SLURM : Lancement de jobs

salloc

Demande d'allocation de ressources

Crée un « job » => assignation d'un ID et de ressources

Shell interactif



SLURM : Lancement de jobs

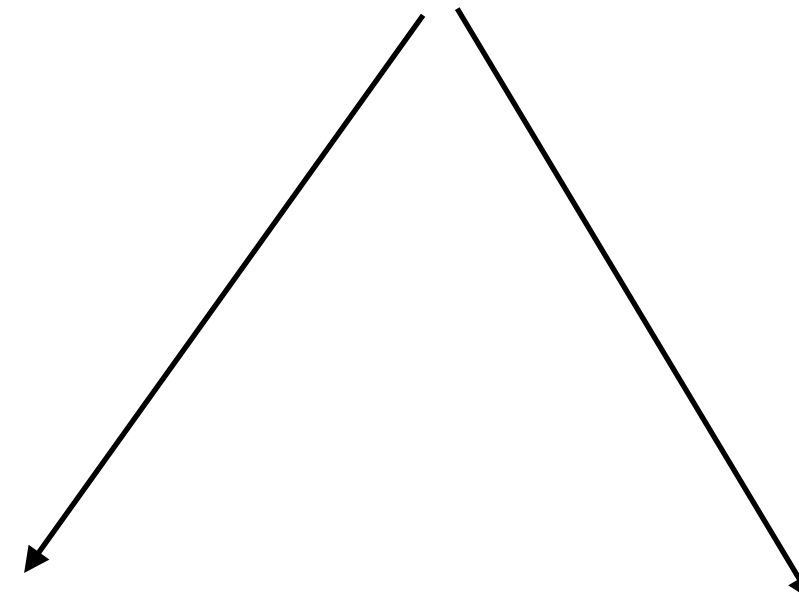
salloc

Demande d'allocation de ressources

Crée un « job » => assignation d'un ID et de ressources

Shell interactif

Exécution locale d'un script



SLURM : Lancement de jobs

salloc : shell interactif

Shell associé aux ressources, accès aux ressources du job

SLURM : Lancement de jobs

salloc : shell interactif

Shell associé aux ressources, accès aux ressources du job

Lancement à la main de steps (srun)

SLURM : Lancement de jobs

salloc : shell interactif

Shell associé aux ressources, accès aux ressources du job

Lancement à la main de steps (srun)

```
> salloc -N1 -n1 -p sandy
```

```
> srun -n1 hostname
```

```
> exit
```

SLURM : Lancement de jobs

salloc : execution d'un script

Préparation d'un script, contenant des steps (srun)

SLURM : Lancement de jobs

salloc : execution d'un script

Préparation d'un script, contenant des steps (srun)

Passage du script au salloc directement

SLURM : Lancement de jobs

salloc : execution d'un script

Préparation d'un script, contenant des steps (srun)

Passage du script au salloc directement

Allocation des ressources, exécution, puis libération

SLURM : Lancement de jobs

salloc : execution d'un script

Préparation d'un script, contenant des steps (srun)

Passage du script au salloc directement

Allocation des ressources, exécution, puis libération

```
>salloc -N1 -n1 -p sandy ./script-1.sh
```

SLURM : Lancement de jobs

sbatch

Demande d'allocation de ressources et création de job

SLURM : Lancement de jobs

sbatch

SLURM : Lancement de jobs

sbatch

Demande d'allocation de ressources et création de job

Soumission **asynchrone** : cycle de vie géré par Slurm

SLURM : Lancement de jobs

sbatch

Demande d'allocation de ressources et création de job

Soumission **asynchrone** : cycle de vie géré par Slurm

Execution du script en argument, comportant des job steps

SLURM : Lancement de jobs

sbatch : exemple de script

```
<script hpl>
```

SLURM : Lancement de jobs

sbatch

Demande d'allocation de ressources et création de job

Soumission **asynchrone** : cycle de vie géré par Slurm

Execution du script en argument, comportant des job steps

Execution du script sur le **premier noeud alloué**

SLURM : Lancement de jobs

Question :

Dans mon script-1.sh :

```
#!/bin/bash
```

```
hostname
```

```
srun hostname
```

SLURM : Lancement de jobs

Question :

Dans mon script-1.sh :

```
#!/bin/bash
```

```
hostname
```

```
srun hostname
```

=> Quelle différence entre

```
salloc -w noeud1 ./script-1.sh
```

SLURM : Lancement de jobs

Question :

Dans mon script-1.sh :

#!/bin/bash

hostname

srun hostname

=> Quelle différence entre

`salloc -w noeud1 ./script-1.sh`

`sbatch -w noeud1 script-1.sh`

SLURM : Lancement de jobs

sbatch

Sorties stdout/stderr redirigé vers un fichier texte

SLURM : Lancement de jobs

sbatch

Sorties stdout/stderr redirigé vers un fichier texte

Soumission **asynchrone** : cycle de vie géré par Slurm

SLURM : Lancement de jobs

sbatch

Sorties stdout/stderr redirigé vers un fichier texte

Soumission **asynchrone** : cycle de vie géré par Slurm

Interaction avec le job avec commandes Slurm:

sinfo, scancel, scontrol, sattach

SLURM : Lancement de jobs

sattach

SLURM : Lancement de jobs

sattach

Suivi et interaction avec un job batch de manière interactive

SLURM : Lancement de jobs

sattach

Suivi et interaction avec un job batch de manière interactive

scancel

SLURM : Lancement de jobs

sattach

Suivi et interaction avec un job batch de manière interactive

scancel

Transmission d'un signal à un job (step)

SLURM : Lancement de jobs

sattach

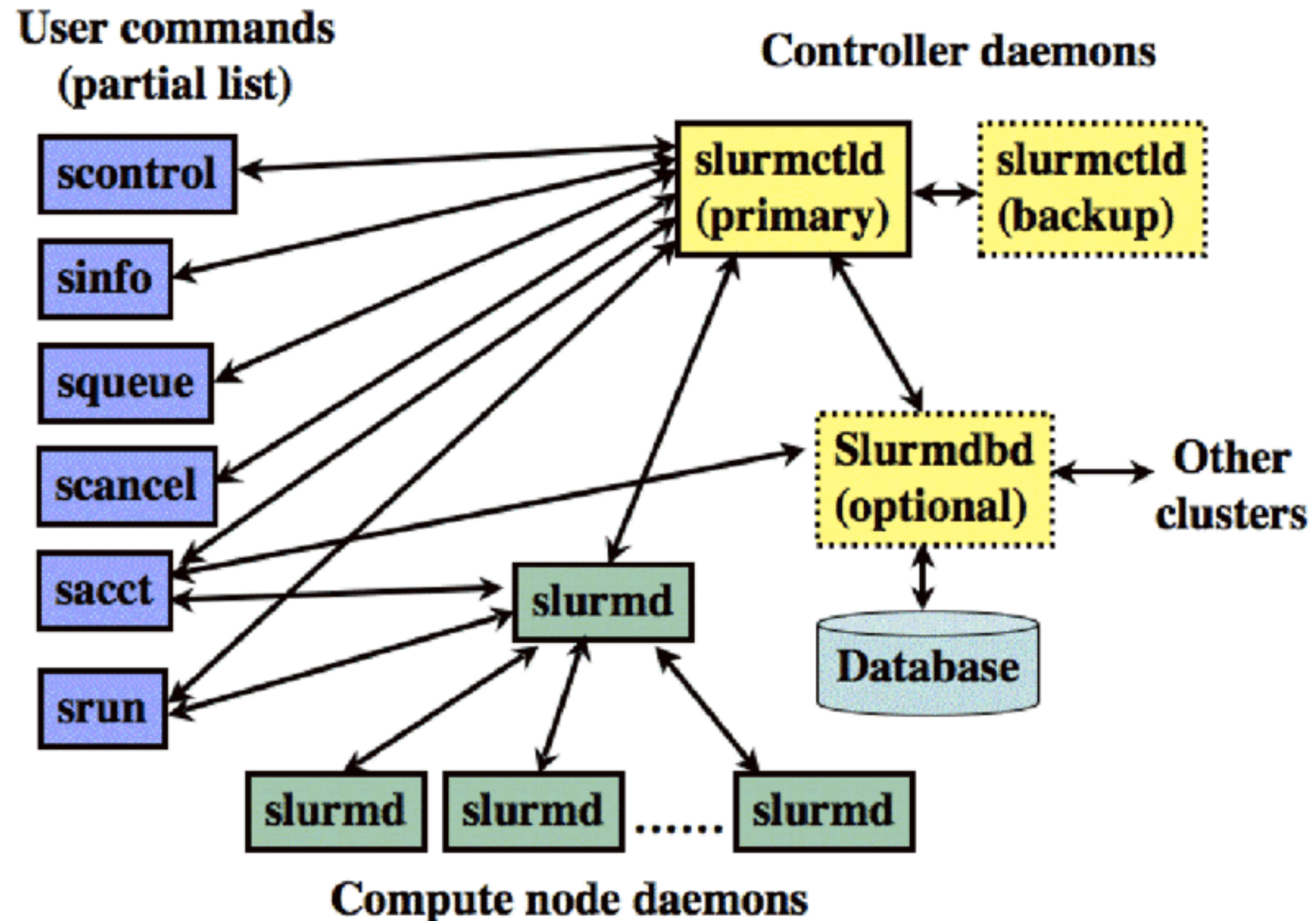
Suivi et interaction avec un job batch de manière interactive

scancel

Transmission d'un signal à un job (step)

Terminaison immédiate d'un job (step)

SLURM : Lancement de jobs



SLURM : Lancement de jobs

Quelques options de lancement

- p : partition
- N : nombre de noeuds
- n : nombre de tâches=process (1/noeud par défaut)
- c : nombre de coeurs (--cpus-per-task)

SLURM : Lancement de jobs

Quelques options de lancement, plus spécifiques...

- cores-per-socket : noeud avec au moins tant de coeurs/sock
- mem-per-cpu
- ntasks-per-node
- exclusive

SLURM : Lancement de jobs

Exemple

`srun -N 1 -n 1 hostname => n1`

`srun -N 1 -n 20 hostname => n1,n1,n1,...`

`srun -N 2 -n 2 hostname => n1,n2`

`srun -N 2 -n 2 -c 2 hostname => n1,n2`

...

Slurm : ordonnancement

SLURM : Ordonnancement

Priority-based scheduling

SLURM : Ordonnancement

Priority-based scheduling

Expression de l'ordre d'exécution en fonction de critères

SLURM : Ordonnancement

Priority-based scheduling

Expression de l'ordre d'exécution en fonction de critères

Assignation d'une **priorité numérique**

SLURM : Ordonnancement

Priority-based scheduling

Expression de l'ordre d'exécution en fonction de critères

Assignation d'une **priorité numérique**

Comparaison numérique simple et rapide

SLURM : Ordonnancement

Priority-based scheduling

Expression de l'ordre d'exécution en fonction de critères

Assignation d'une **priorité numérique**

Comparaison numérique simple et rapide

e.g. : execution de celui avec priorité + grande d'abord

SLURM : Ordonnancement

Politiques d'ordonnancement

SLURM : Ordonnancement

Politiques d'ordonnancement

Organisation dans le temps de l'exécution des jobs

SLURM : Ordonnancement

Politiques d'ordonnancement

Organisation dans le temps de l'exécution des jobs

Se base sur la priorité numérique

SLURM : Ordonnancement

Politiques d'ordonnancement

Organisation dans le temps de l'exécution des jobs

Se base sur la priorité numérique

Détermine qui dans la queue passe le prochain

SLURM : Ordonnancement

Politiques d'ordonnancement : FIFO

SLURM : Ordonnancement

Politiques d'ordonnancement : FIFO

First-IN First-OUT

SLURM : Ordonnancement

Politiques d'ordonnancement : FIFO

First-IN First-OUT

Par ordre de rentrée

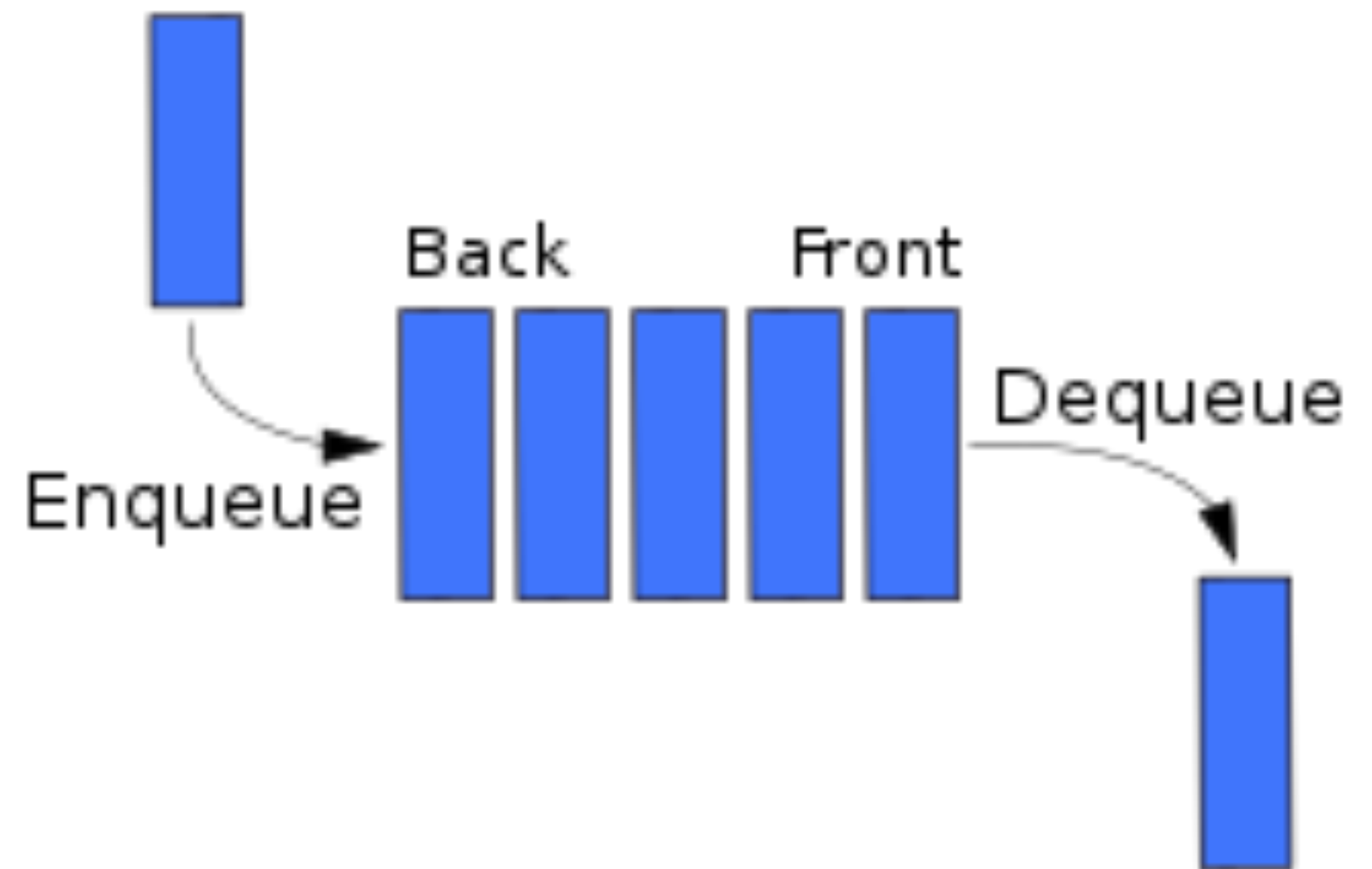
SLURM : Ordonnancement

Politiques d'ordonnancement : FIFO

First-IN First-OUT

Par ordre de rentrée

Très (trop) basique...



SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling

SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling

Un job moins prioritaire est exécuté en premier s'il ne repousse pas la date de démarrage des plus prioritaires

SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling

*Un job moins prioritaire est exécuté en premier s'il
ne repousse pas la date de démarrage des plus prioritaires*

=> combler les trous

SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling

Un job moins prioritaire est exécuté en premier s'il ne repousse pas la date de démarrage des plus prioritaires

=> combler les trous

=> faire passer des jobs moins prioritaires et petits plus vite

SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling

Liste des jobs en attente

SLURM : Ordonnancement

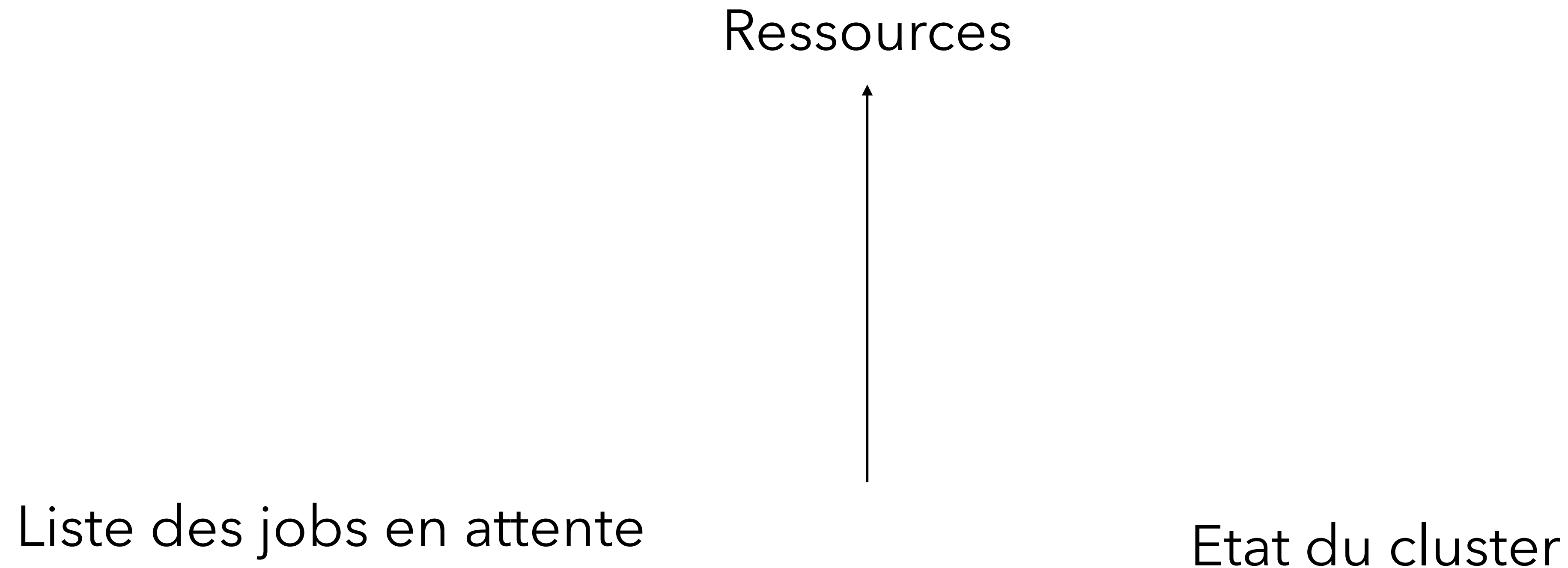
Politiques d'ordonnancement : Backfilling

Liste des jobs en attente

Etat du cluster

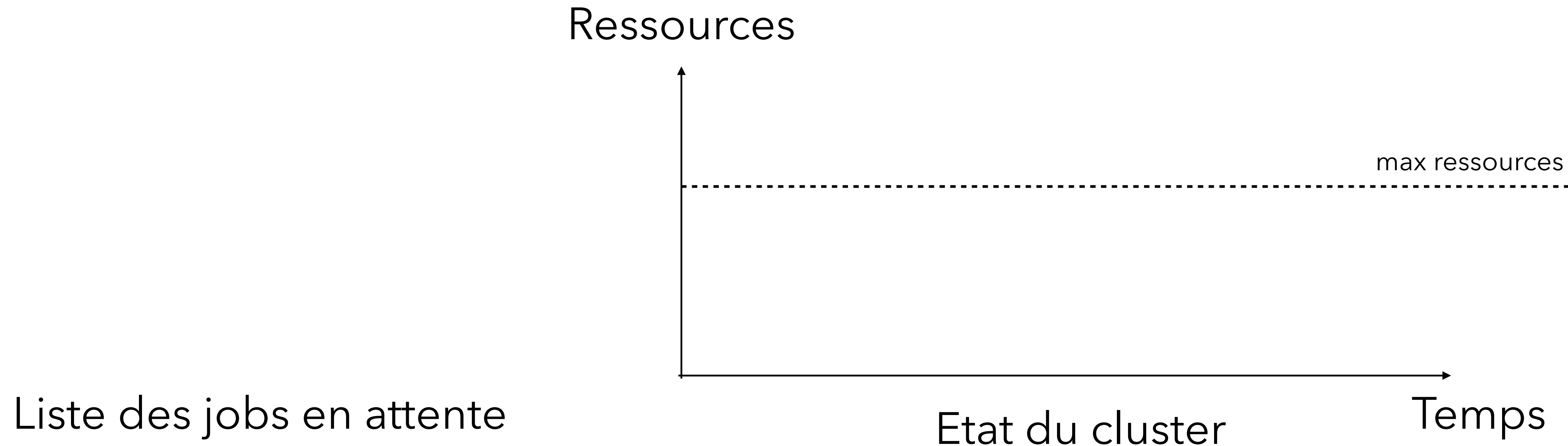
SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling



SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling



SLURM : Ordonnancement

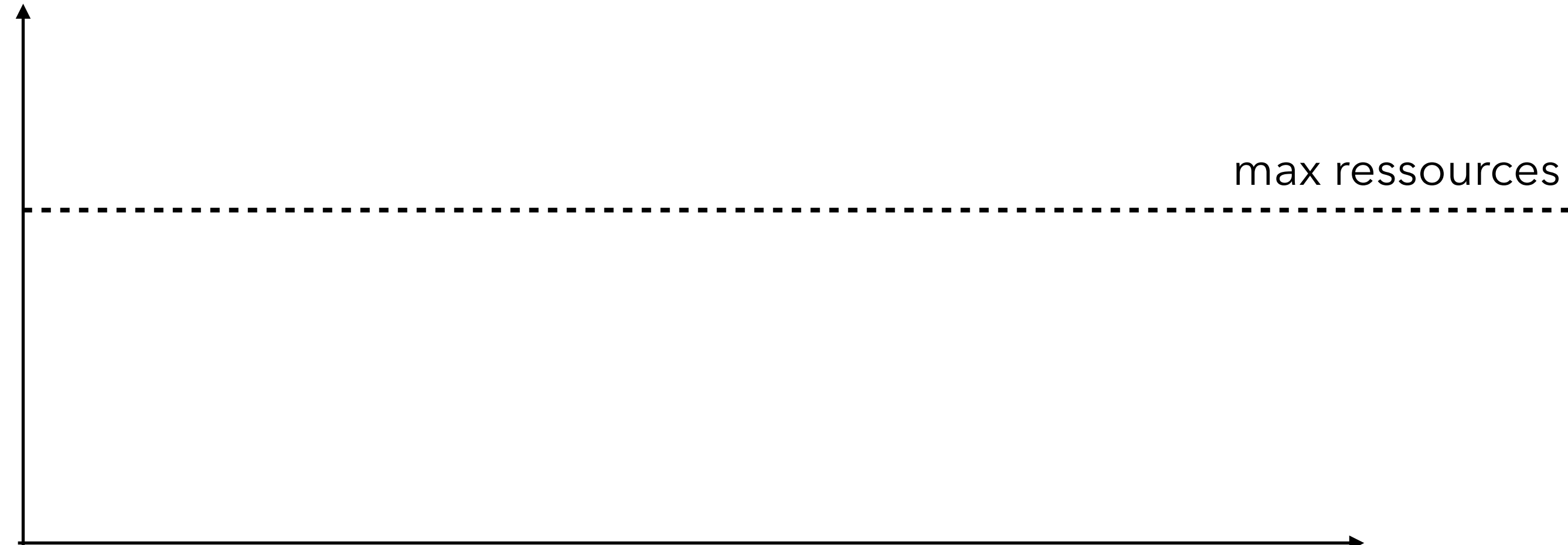
Politiques d'ordonnancement : Backfilling

1

Prio 100

Liste des jobs en attente

Ressources

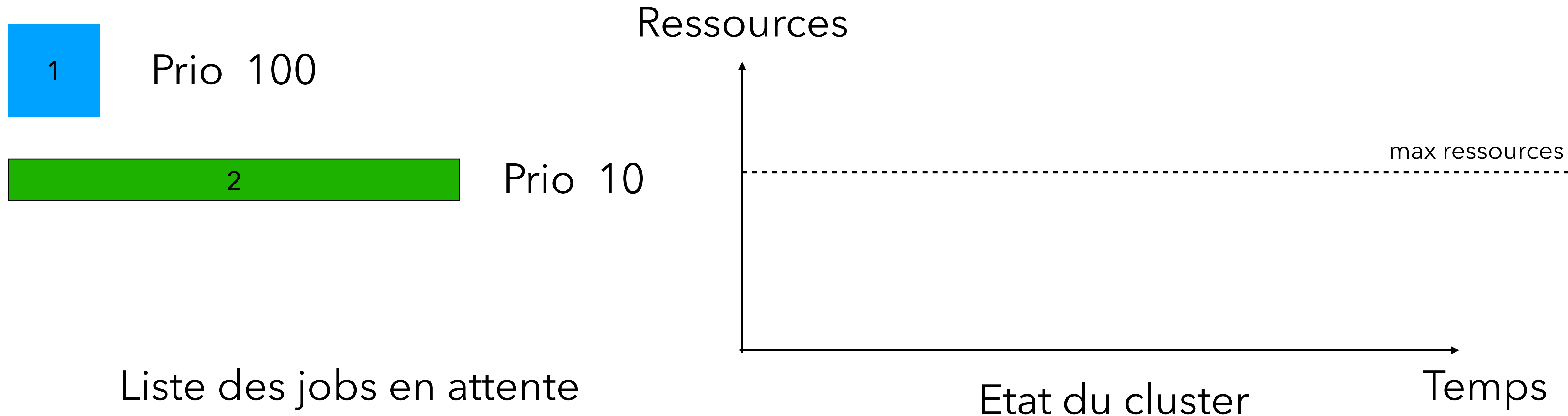


Etat du cluster

Temps

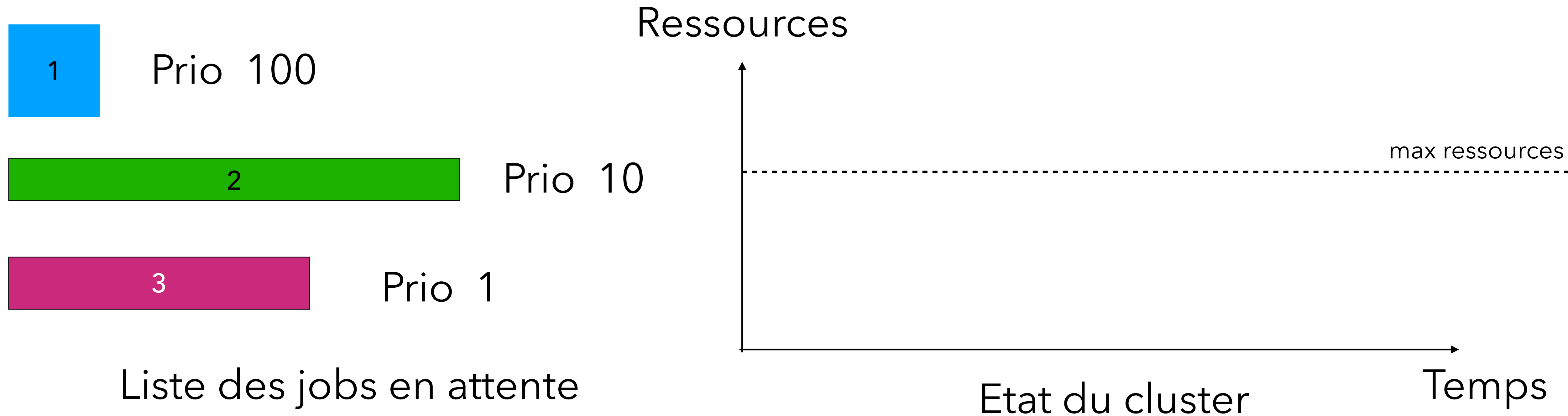
SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling



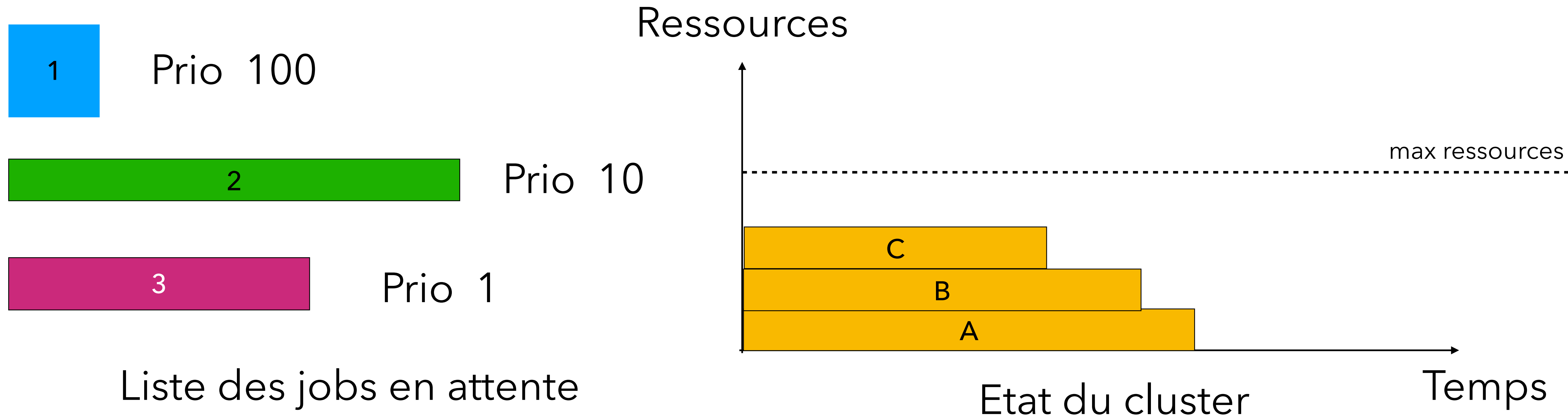
SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling



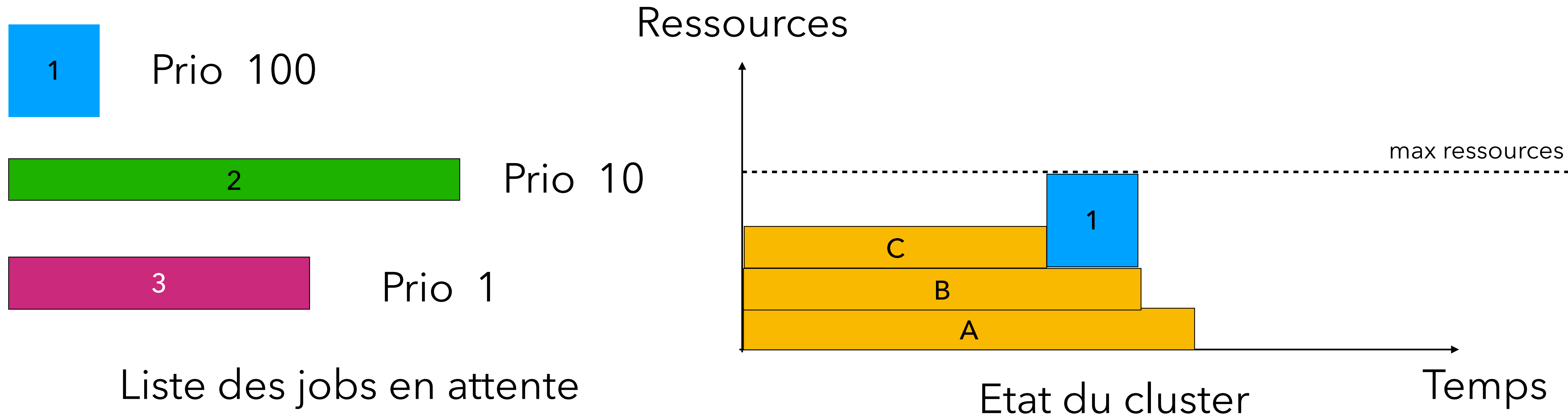
SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling



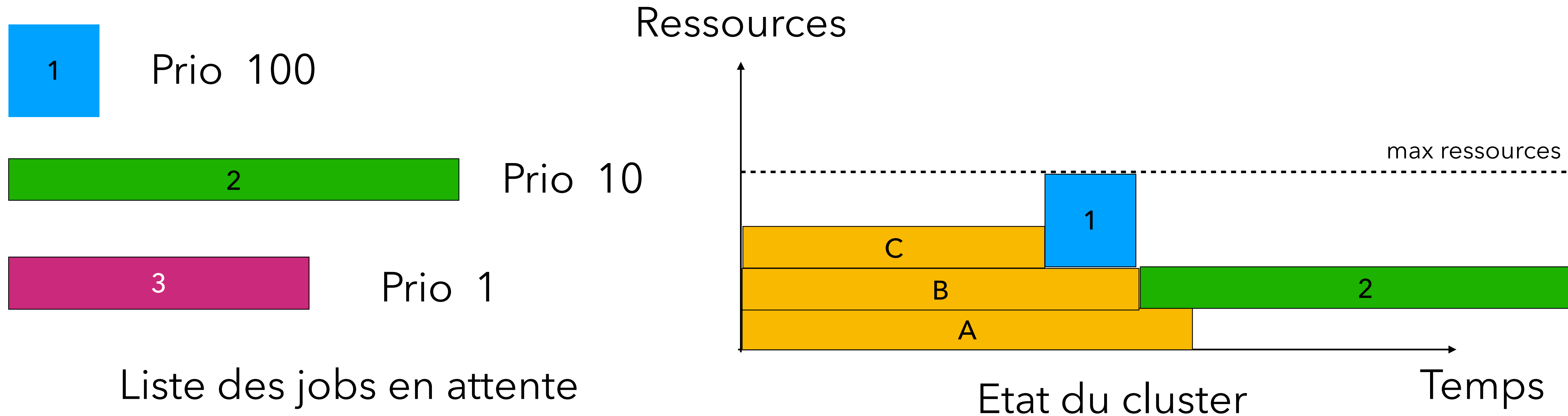
SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling



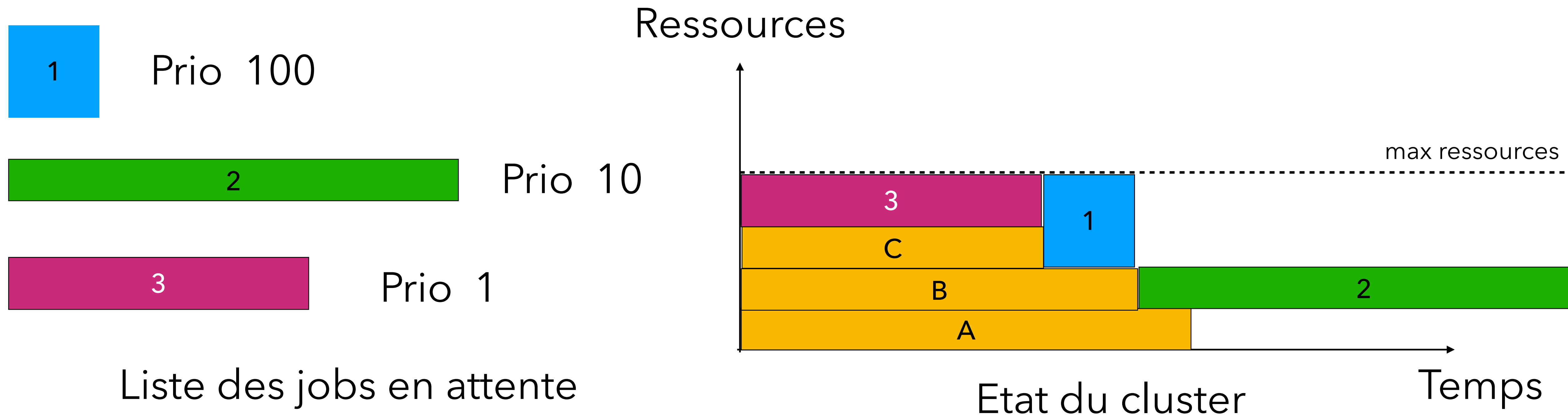
SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling



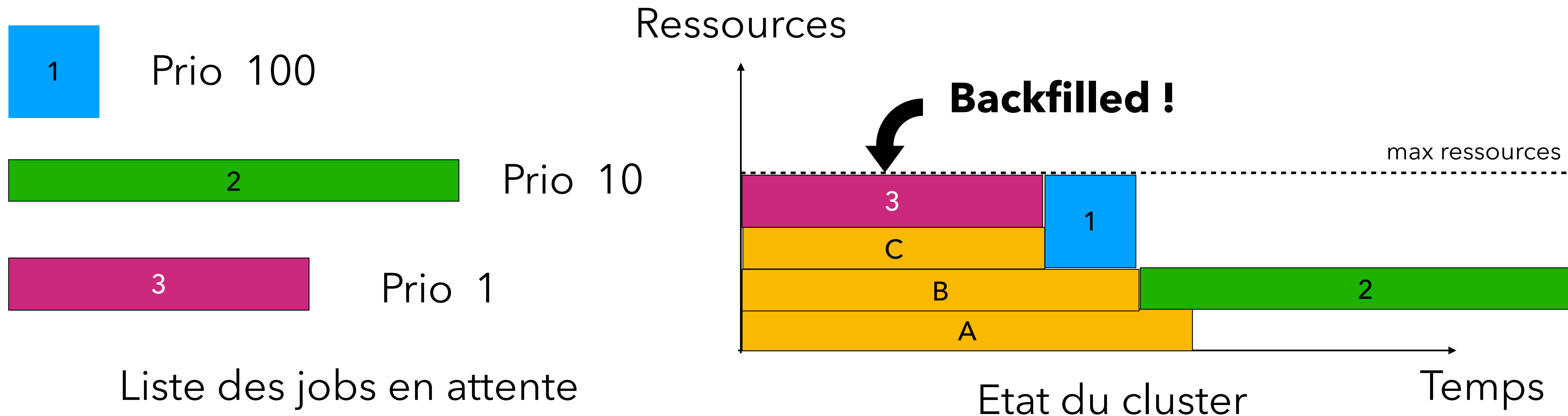
SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling



SLURM : Ordonnancement

Politiques d'ordonnancement : Backfilling

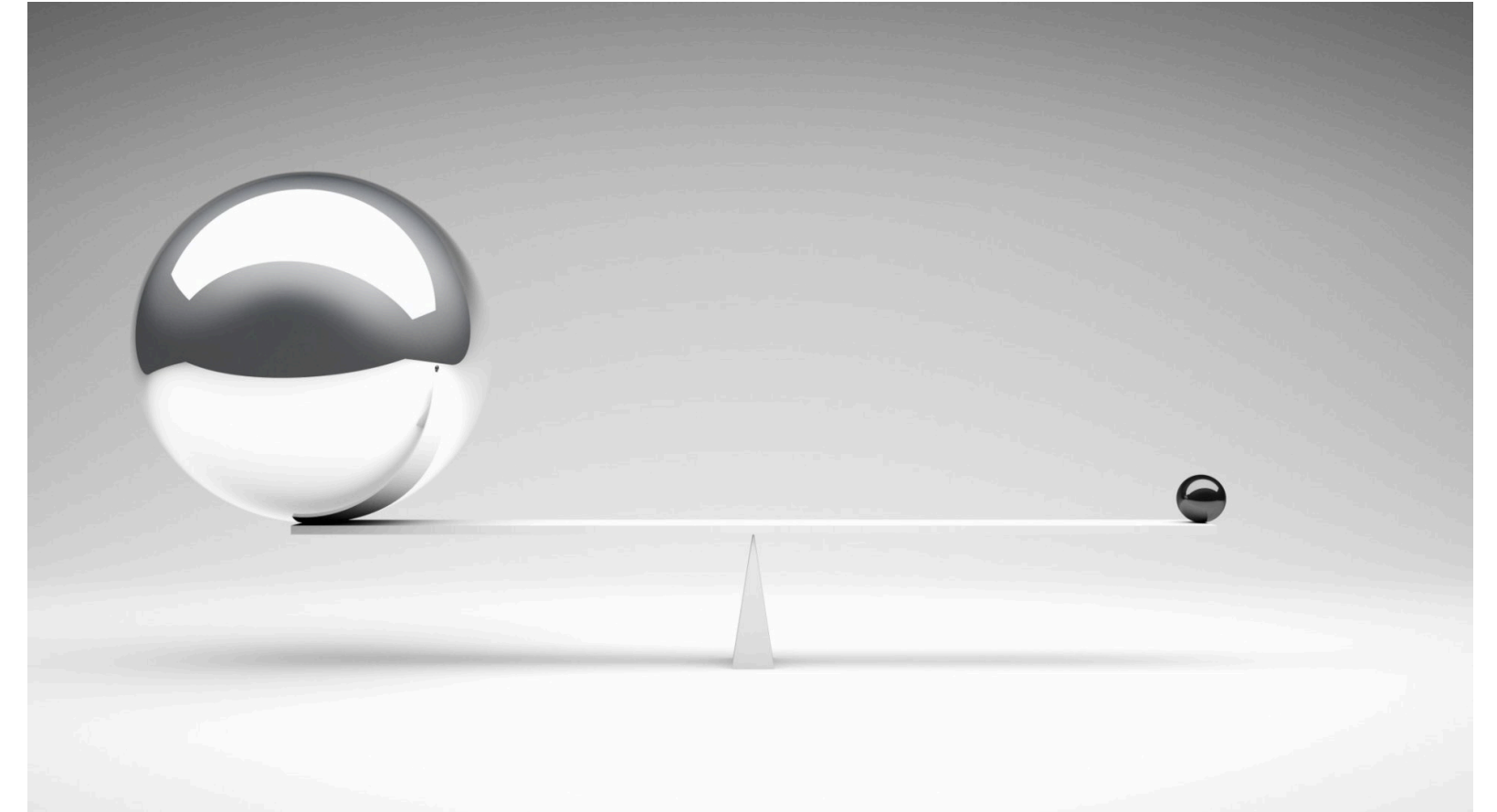


SLURM : Ordonnancement

Politiques de calcul des priorités

Basé sur la **taille** (size based)

QOS : quality of service



IMPORTANT

SLURM : Ordonnancement

Politiques de calcul des priorités

Fairshare :

Parts de ressources attribuées

Equal Share



Fair Share



SLURM : Ordonnancement

Politiques de calcul des priorités

Facteur âge :

Basé sur le temps d'attente

Evite la « famine » (starvation)



SLURM : Ordonnancement

Politiques de calcul des priorités

Facteur partition/QOS

Priorité pour chaque partition/QOS

Normalisation de la valeur de la partition par rapport
à la priorité maximale observée

SLURM : Ordonnancement

Politiques de calcul des priorités

Facteur partition/QOS

Partition B ; priorité = 100 $\Rightarrow$ facteur = 1

Partition A ; priorité = 20 $\Rightarrow$ facteur = 0.2

Partition C ; priorité = 70 $\Rightarrow$ facteur = 0.7

SLURM : Ordonnancement

Politiques de calcul des priorités

Fairshare

Ecart entre part utilisable et utilisée

Idée : maintenir à l'équilibre l'utilisation

Niveau utilisateur, groupe d'utilisateurs, etc...

SLURM : Ordonnancement

Politiques de calcul des priorités

Fairshare

Utilisateur A : $\text{share} = 0.3$; $\text{usage} = 0.2 \Rightarrow \text{facteur} = 0.6$

Utilisateur B : $\text{share} = 0.2$; $\text{usage} = 0.25 \Rightarrow \text{facteur} = 0.45$

SLURM : Ordonnancement

Politiques de calcul des priorités

Priorité **Multifactorielle**

Mélange de différentes politiques

Facteur de pondération

9 facteurs : Age, Association, Fairshare, Taille, Nice, Partition,
QOS, Site, TRES

SLURM : Ordonnancement

Politiques de calcul des priorités

Priorité **Multifactorielle**

Facteurs entre 0 et 1

```
Job_priority =  
    site_factor +  
    (PriorityWeightAge) * (age_factor) +  
    (PriorityWeightAssoc) * (assoc_factor) +  
    (PriorityWeightFairshare) * (fair-share_factor) +  
    (PriorityWeightJobSize) * (job_size_factor) +  
    (PriorityWeightPartition) * (priority_job_factor) +  
    (PriorityWeightQOS) * (QOS_factor) +
```

SLURM : Ordonnancement

PriorityType=priority/multifactor

PriorityWeightQOS=100 000

PriorityWeightAge=10 000

PriorityWeightFairshare=10 000

PriorityWeightJobSize=0

PriorityWeightPartition=0

SLURM : Ordonnancement

QOS - Quality of Service

Limitation de l'utilisation des ressources/user (groupe)

Restriction **accès, quantité, temps d'utilisation**

Contrôle plus fin que partition simple

Accès d'une même partition via différentes QOS

SLURM : Ordonnancement

QOS - Limites

MaxJobsPerUser

MaxSubmitJobsPerUser

MaxNodes

MaxWall

....

SLURM : Ordonnancement

QOS - Création

#sacctmgr create qos debug set priority 100

MaxJobsPU=2 MaxWall=01:00:00

#sacctmgr create qos normal set priority 50

MaxJobsPU=10 MaxWall=08:00:00

SLURM : Ordonnancement

Association

4 paramètres : **user, cluster, partition, account**

Application hiérarchique sur les associations

Autant d'associations que nécessaire

Utilisateur dans plusieurs comptes, etc...

SLURM : Ordonnancement

Association - Exemples de limites

DefaultQOS

MaxJobs

GrpJobs

GrpWall

Fairshare {=parent}

SLURM : Ordonnancement

Association - Exemple

```
#sacctmgr add user gaudinr account=root qos=normal
```

```
#sacctmgr modify user gaudinr set qos=test,debug
```

```
#sacctmgr show assoc
```

```
$srun -n 1 --qos=debug -p p1 ~user/myprogdebug
```

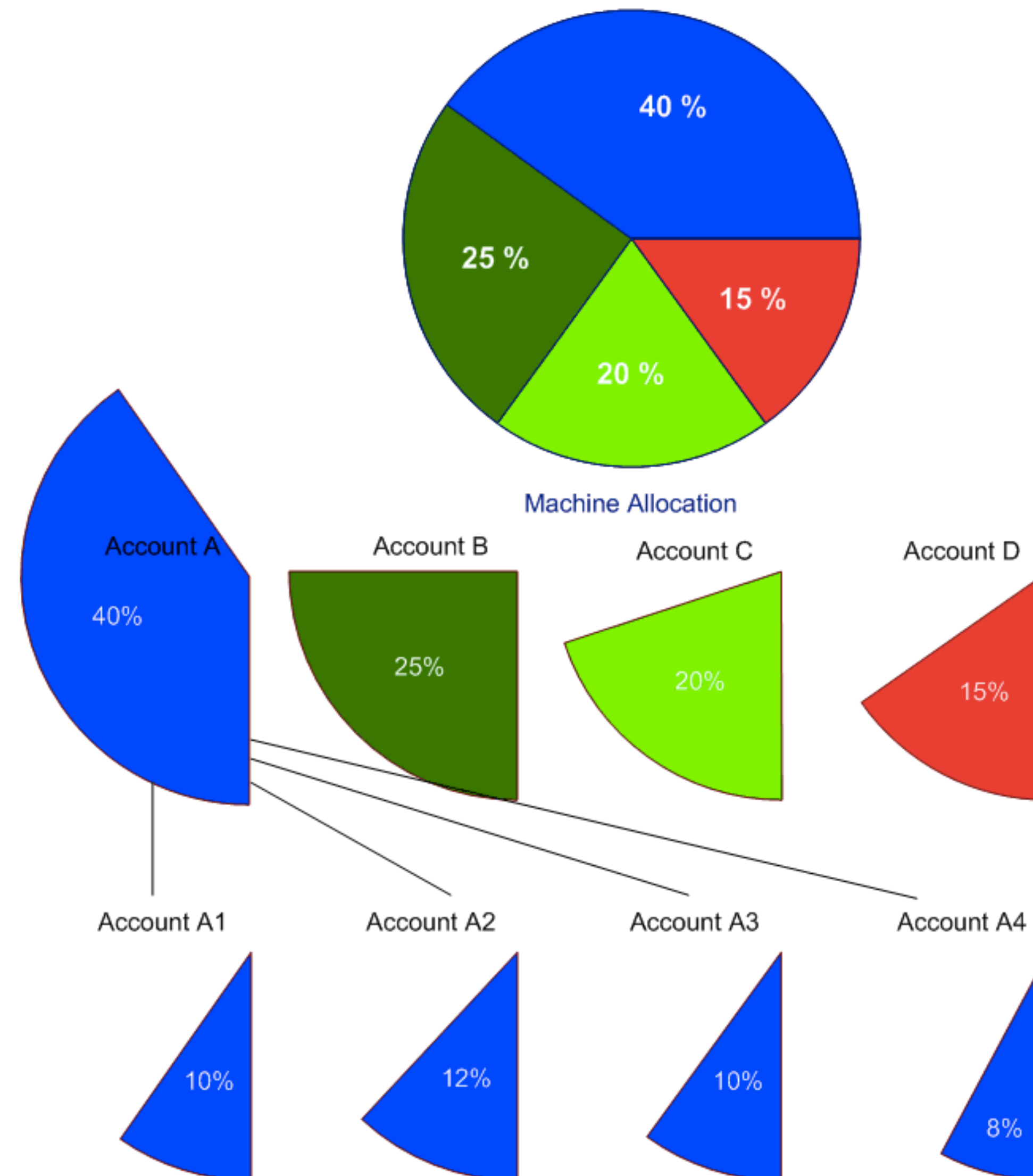
SLURM : Ordonnancement

Fairshare

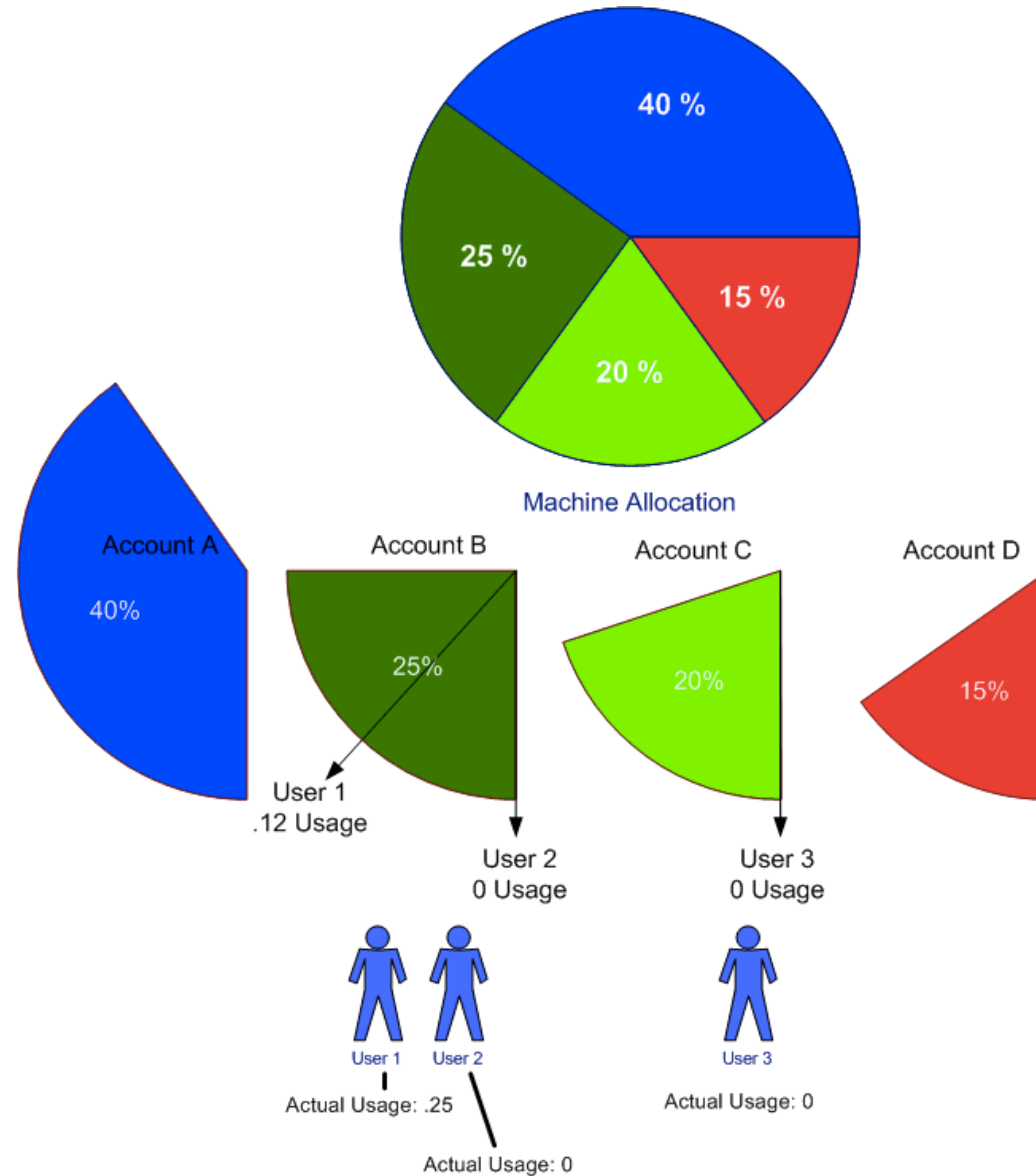
Groupelement hiérarchique d'utilisateurs

Account : traçage de l'utilisation de ressources pour un
groupe d'utilisateurs

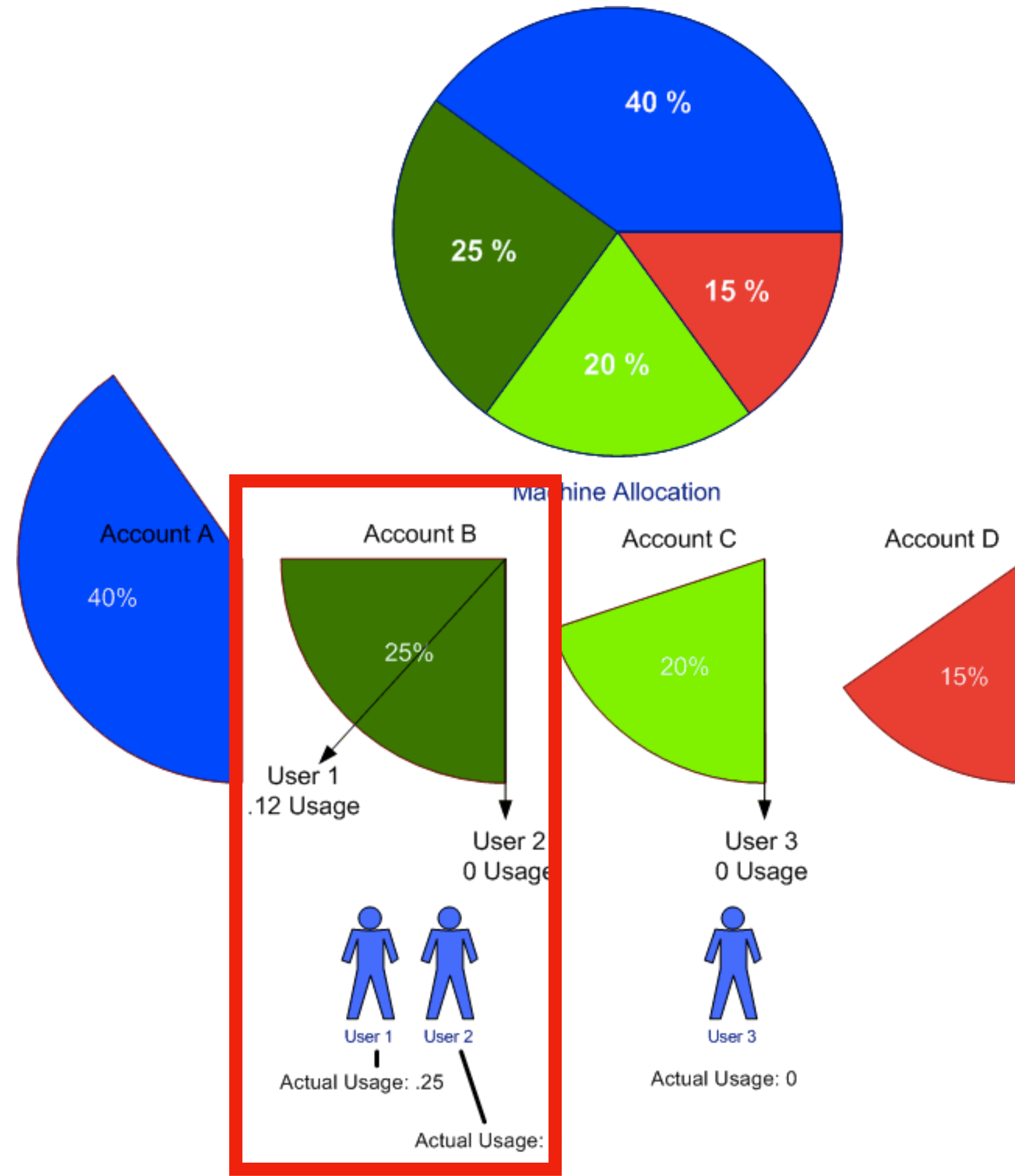
SLURM : Ordonnancement



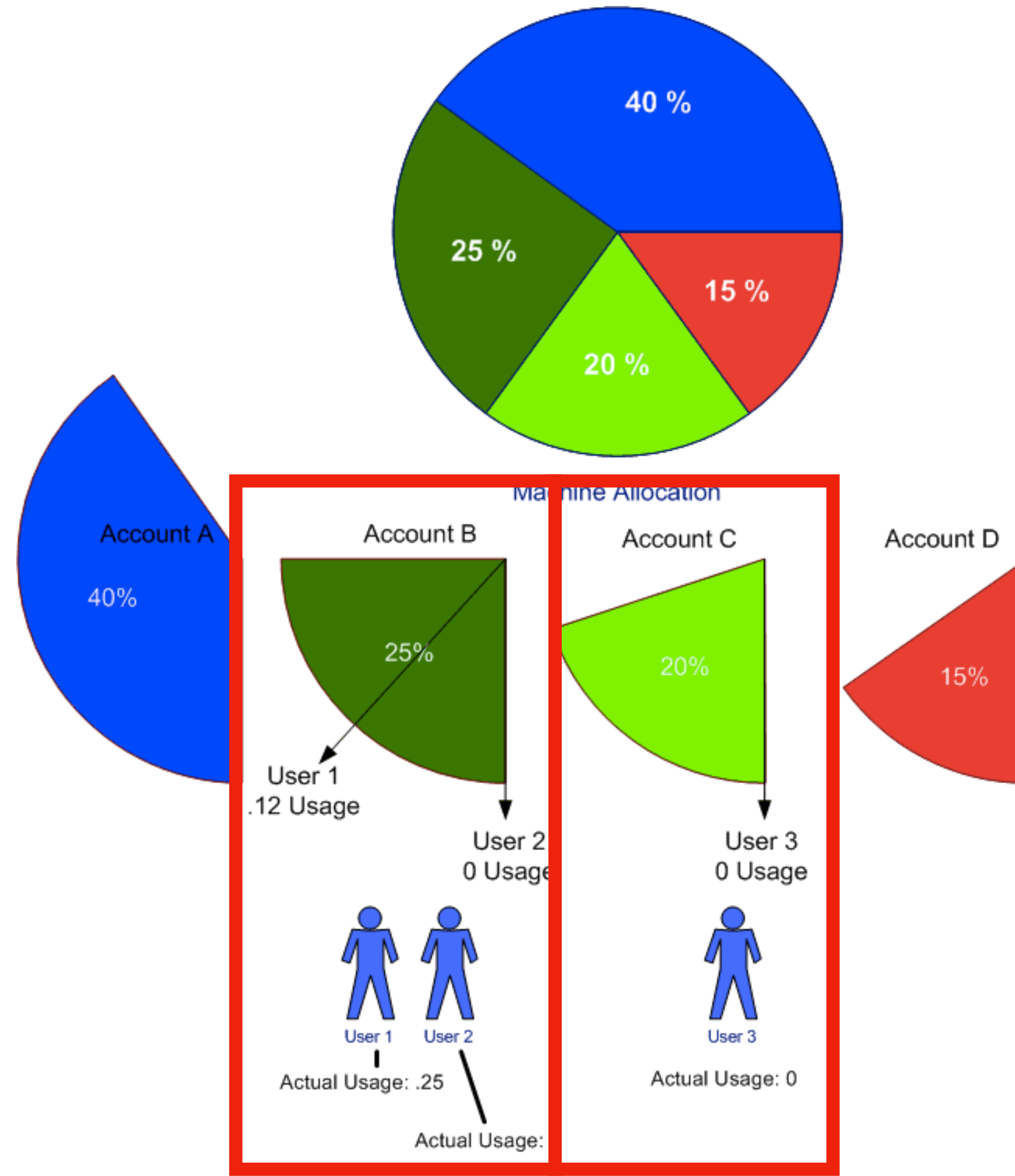
SLURM : Ordonnancement



SLURM : Ordonnancement

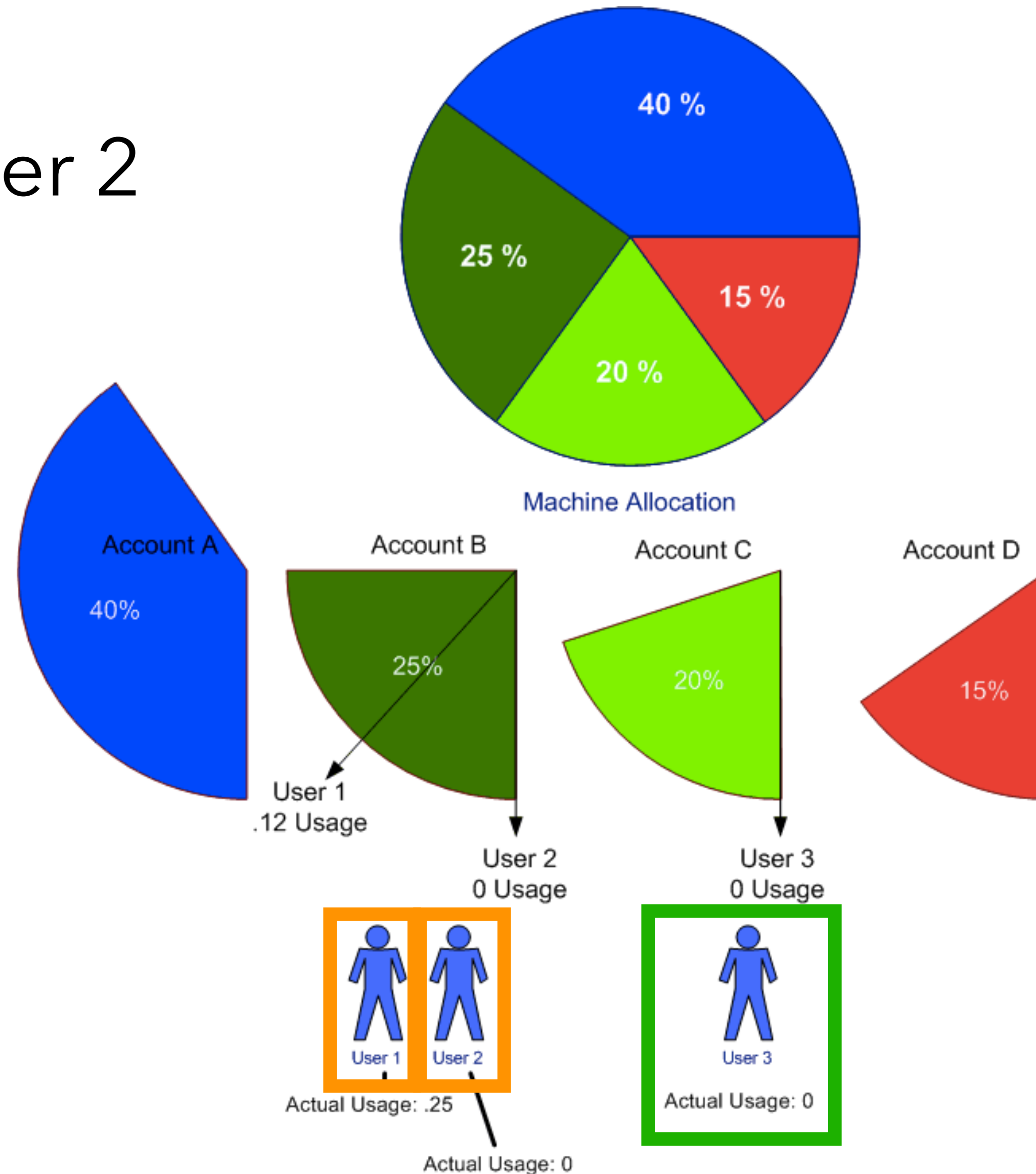


SLURM : Ordonnancement



SLURM : Ordonnancement

- User 1 impacte User 2
- User 3 priorisé



SLURM : Ordonnancement

Association - Création du fair share

```
sacctmgr create cluster=toto
```

```
sacctmgr create account name=economie fairshare=40
```

```
sacctmgr create account name=science fairshare=60
```

```
sacctmgr create account name=chimie parent=science fairshare=40
```

```
sacctmgr create account name=physique parent=science fairshare=20
```

```
sacctmgr user name=kevin cluster=toto account=chimie
```

SLURM : Ordonnancement

=> https://slurm.schedmd.com/classic_fair_share.html

SLURM : Ordonnancement

Fairshare - Conclusions

Certains utilisateurs tournent trop de jobs : facteur < 0.5

Certains utilisateurs n'ont pas tourné de jobs, leur facteur est proche de 1 et renforcé (pas 0.5)

Certains utilisateurs peuvent avoir une priorité faible, alors qu'ils n'ont pas tourné de job!

=> Problème du fairshare par account : les collègues!

SLURM : Conclusion

Toutes les infos :

=> **<https://www.schedmd.com/>**

Slurm 23 et au delà

SLURM 23

Mises à jour fréquentes

« Major release » tous les 9 mois : ANNEE.MOIS (23.11)

Quelques features...

SLURM 23

Conteneurs

Lancement direct de conteneurs OCI : container **runtime**

Commande « scrun », indirecte

Utilisable par Docker/Podman, de manière transparente

=> Execution du conteneur sur noeud de calcul

SLURM 23

Gestion des ressources

Toujours plus précis : gestion par **tâches**

Amélioration des **réservations**

Commentaires, ajout/retrait de noeuds dynamiquement

Nouvel état pour un job : **FailedNode**

Performance : moins d'échanges d'authentification

SLURM 23

Nombre de noeuds plus flexible

Min, max, liste de nombre acceptable

Allocation par palier (`-nodes=10-30:5`)

Modification à la volée du niveau de debug

Limitation des RPC clients (surcharge `slurmctld`)

SLURM 24 (mai 2024)

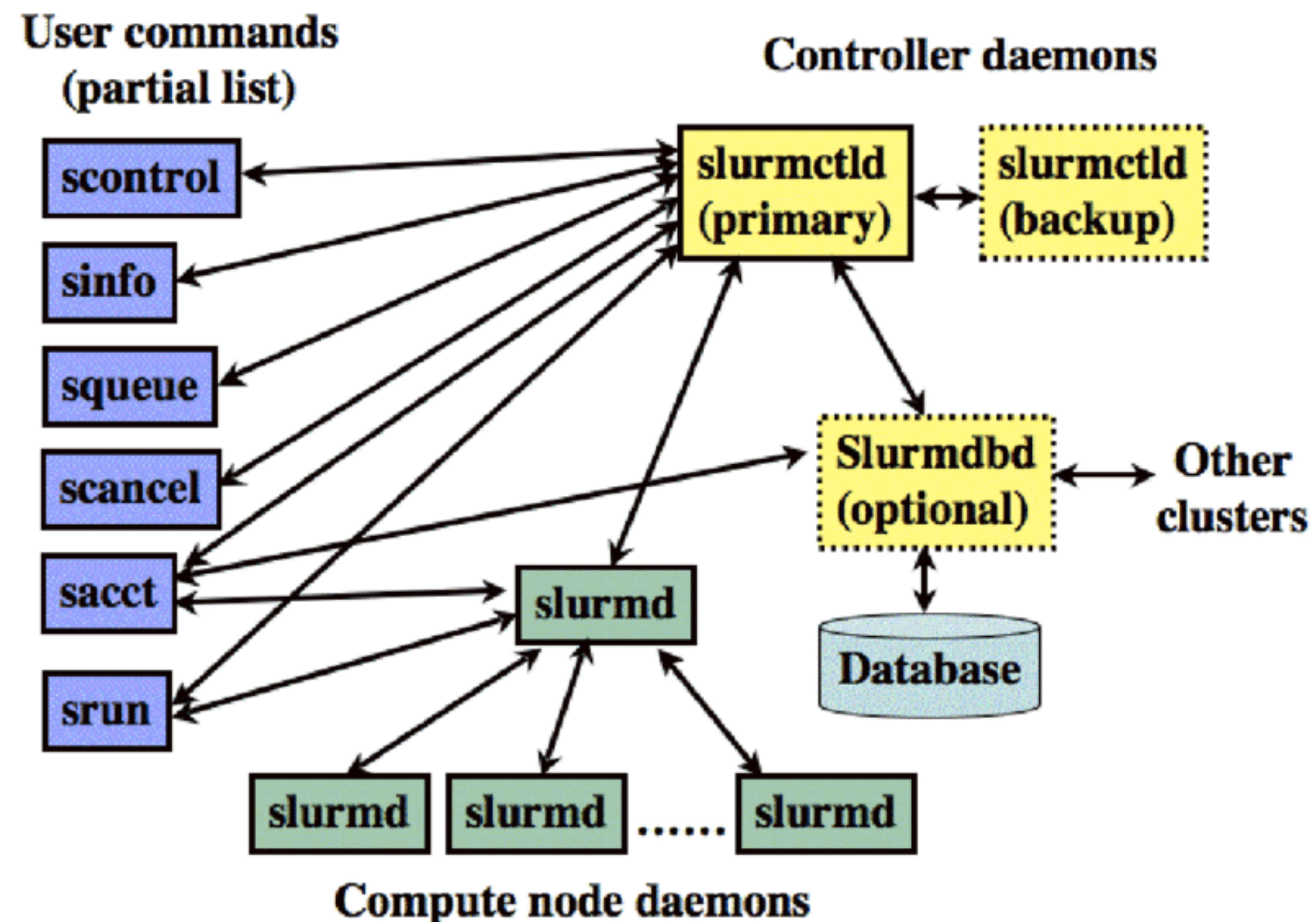
Soumission de jobs avec plusieurs QOS

Commence par la plus prioritaire

Gestion des steps par le démon local, et pas slurmctld

SLURM 24 (mai 2024)

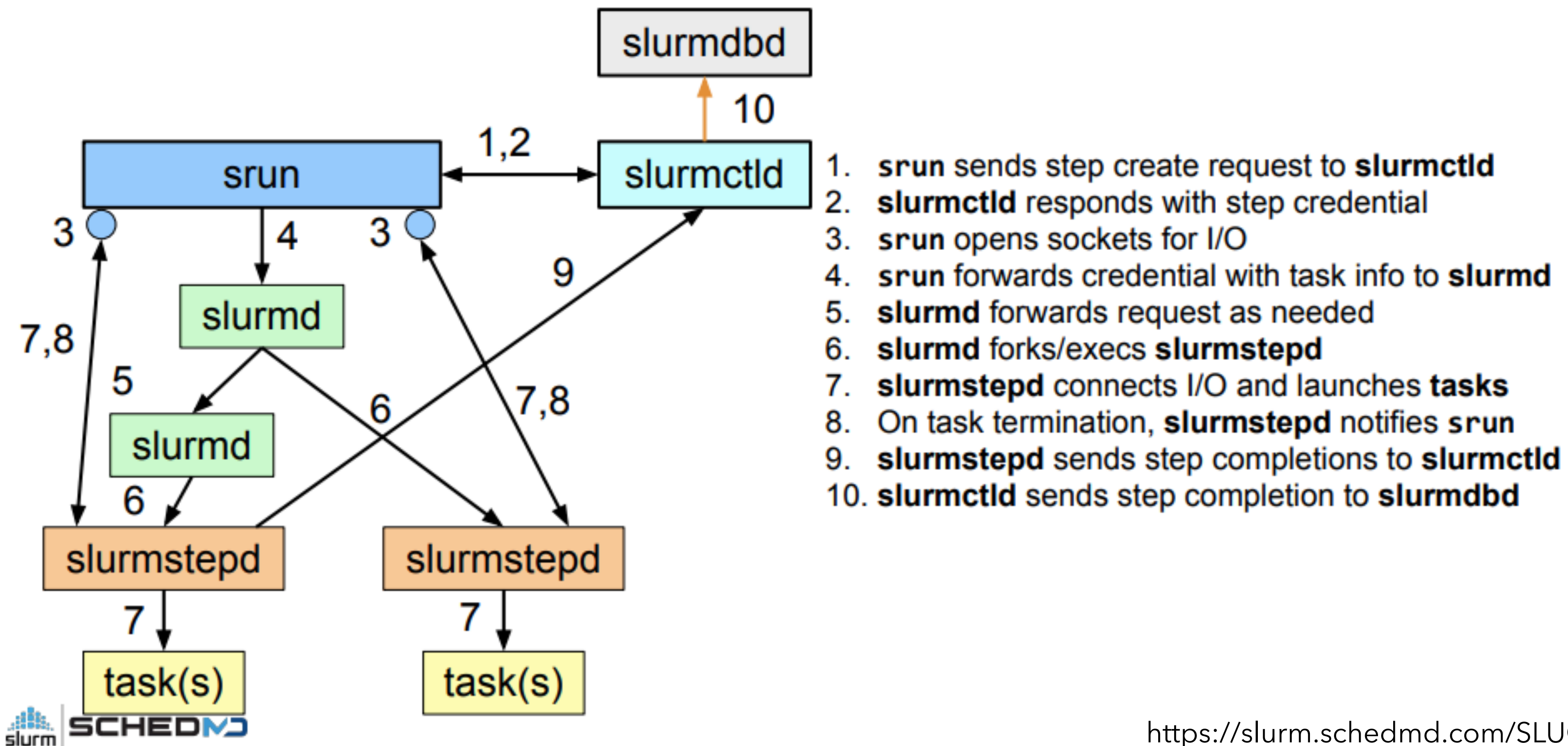
Gestion des steps par le démon local, et pas slurmctld



SLURM 24 (mai 2024)

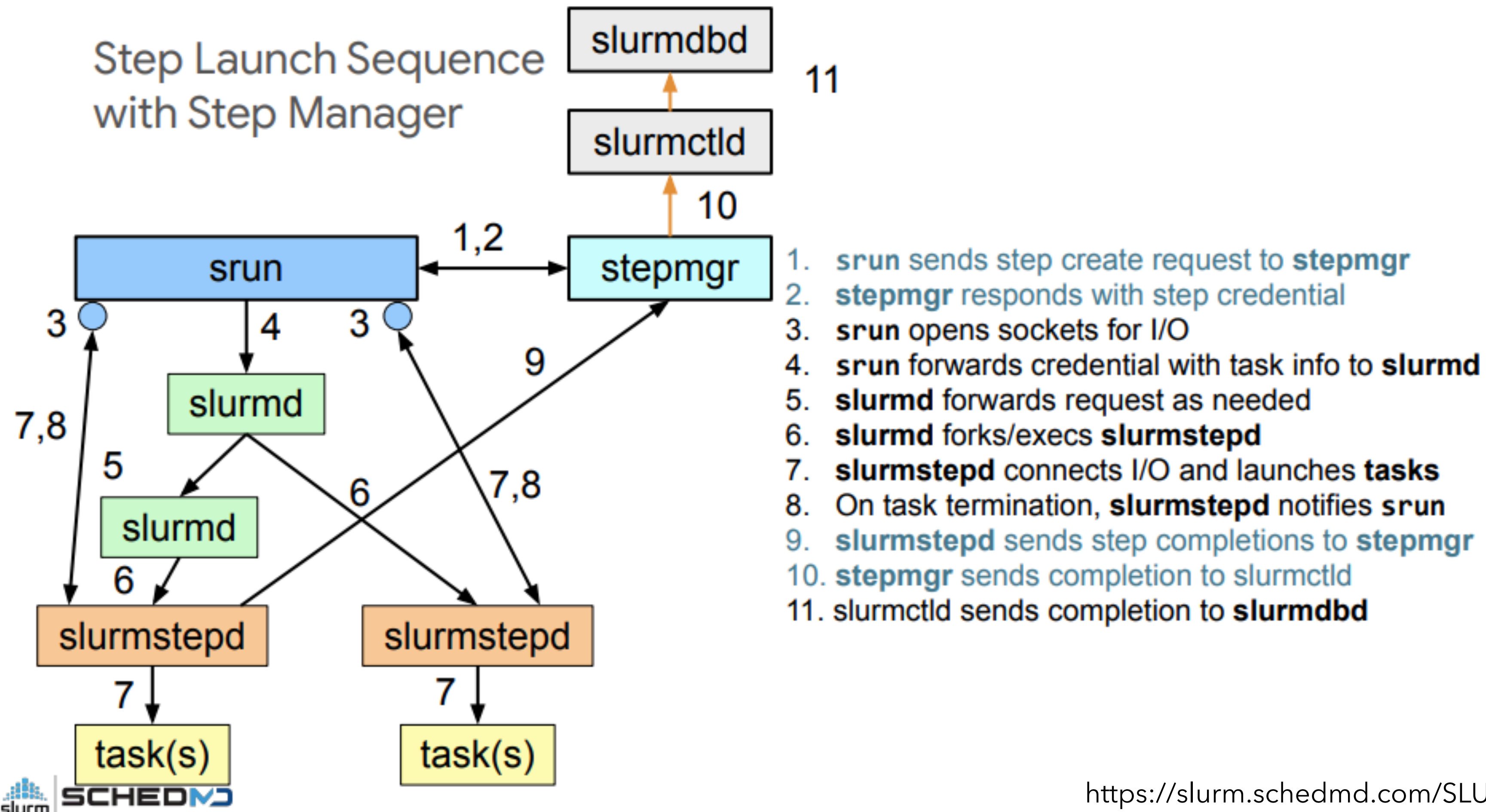
Gestion des steps par le démon local, et pas slurmctld

Step Launch Sequence



SLURM 24 (mai 2024)

Gestion des steps par le démon local, et pas slurmctld



SLURM 25 (mai 2025)

Amélioration gestion mémoire dans le contrôleur

Ajout de noeud dynamiquement dans la topologie

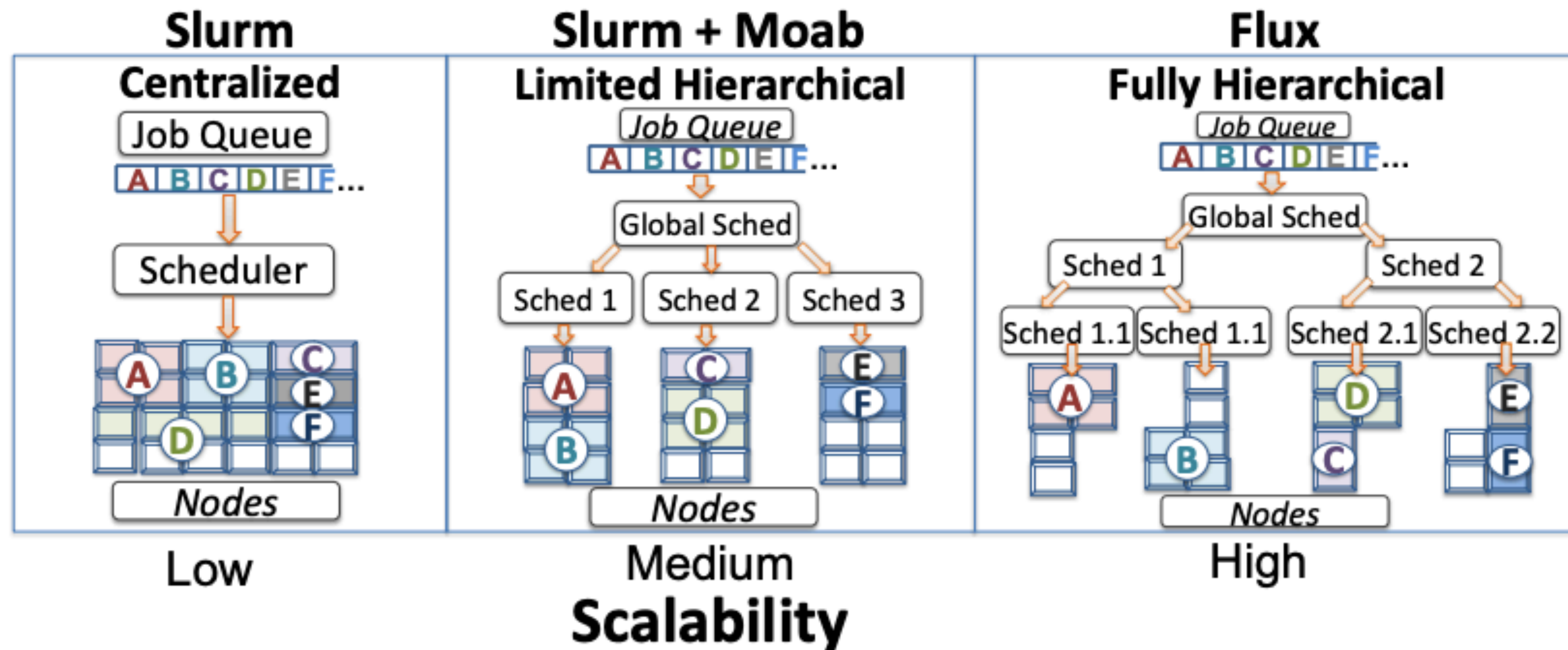
Support chiffrement TLS pour toutes les communications

Ajout d'une fonctionnalité à l'API pour créer des réservations

Autre architecture : Flux

Flux

A flexible framework for resource management customized for your HPC site



Flux

Installation en tant que scheduler unique

Utilisation directement dans Slurm

Allocation de ressources dans un job

Ressources du job gérées par Flux

=> Décharge le contrôleur

Partie 2...